ARISTOTLE UNIVERSITY OF THESSALONIKI

SCHOOL OF INFORMATICS

BACHELOR THESIS

Runtime Verification of Autonomous Driving Systems in CARLA

Επαλήθευση συστημάτων αυτόνομης οδήγησης σε χρόνο εκτέλεσης στο περιβάλλον προσομοίωσης **CARLA**

Author:

Eleni ZAPRIDOU
Ελένη Ζαπρίδου
AEM: 2937

Supervisors:

Prof. Panagiotis KATSAROS
Aristotle University of Thessaloniki

Prof. Ezio BARTOCCI
Technical University of Vienna

July, 2020

Summary

Autonomous vehicles are likely to run on cleaner, electric powertrains, and they should also, in theory, be much safer drivers than today's human operators. However, getting to the point where this is the norm will require a significant expansion of current testing deployments. Although real-world testing of autonomous vehicles has increased in recent years, the industry is still far from producing a fully autonomous vehicle. As such, the key element in pushing the limits of today's self driving cars lies not in physical testing, but in the trials and pilots conducted in the virtual world.

In part, this is due to practical limitations. Opting for initial virtual testing can enable the design exploration at the early system development stages, reducing high infrastructure costs and high risks. It can also allow developers to quickly iterate and implement new algorithms and feed in new test data. However, of particular value is the opportunity to use the virtual world to teach AI to think like a human. Urban driving simulators, such as CARLA, provide 3-D environments and useful tools to easily simulate sensorimotor control systems in scenarios with complex multi-agent dynamics.

Even so, due to the high-dimensional input and state spaces of closed-loop autonomous driving systems, the testing and verification of such simulators is very challenging. A promising avenue to address this challenge is the prospect to invest on the recent developments in theory and tools for runtime verification. Runtime monitoring continuously checks the system's performance by detecting if it is consistent with a high-level specification of requirements, usually given as temporal logic formulas, for example expressed in Signal Temporal Logic (STL). Additional advances on runtime verification of STL enable the systematic design space exploration.

The current thesis aims to explore and present the functionalities of CARLA in controlling the simulator's actors for the development of autonomous systems, customizing the environment to achieve diverse scenarios, achieving reproducibility of the test cases and collecting simulation traces. Secondly, we show here how to integrate the recently introduced `rtamt` library, for runtime verification of STL specifications, with the CARLA simulator. Finally, we also present the obtained results from exploring the design space of an experimental Adaptive Cruise Control system, in two variants, with respect to various PID control parameters and monitoring quantitatively interesting requirements for the system tested in CARLA.

Περίληψη

Παρότι η ερευνητική κοινότητα και οι κατασκευαστές αυτοκινούμενων οχημάτων δεν έχουν πετύχει ακόμη την καθολικά αυτόνομη οδήγηση, έχουν καταφέρει σημαντικά βήματα. Σήμερα, υπάρχουν ημιαυτόνομα οχήματα με τεχνολογίες υποβοήθησης της οδήγησης που διευκολύνουν τον οδηγό και βελτιώνουν την προσοχή του. Μετά τις εξελίξεις αυτές, οι κυριότερες προκλήσεις τώρα έγκεινται στην έλλειψη ασφάλειας και στο υψηλό κόστος και κίνδυνο της διεξαγωγής των δοκιμών τέτοιων συστημάτων. Στο πλαίσιο αυτό, είναι σημαντικό να γνωρίζουμε όχι μόνο ποιες είναι οι ελάχιστες απαιτήσεις, που πρέπει να ικανοποιούνται, και πώς μπορούμε να τις επαληθεύσουμε, αλλά και υπό ποιες συνθήκες οι απαιτήσεις αυτές τείνουν να παραβιαστούν.

Οι δοκιμές σε πραγματικό περιβάλλον συχνά απαγορεύονται και απαιτούν ακριβό εξοπλισμό και υπερβολικά μεγάλο χρόνο. Επίσης, στο φυσικό κόσμο δύο δοκιμές δεν μπορούν ποτέ να επαναλάβουν ακριβώς τις ίδιες συνθήκες, δεδομένης της μεταβλητότητας του καιρού, της συμπεριφοράς των άλλων οχημάτων και των πεζών κλπ. Για να διασφαλιστεί ότι ένα αυτο-οδηγούμενο όχημα είναι ασφαλές και για να μειώσουμε τα κόστη των δοκιμών, πλήθος σεναρίων μπορεί να μοντελοποιηθεί και να εκτελεστεί σε προσομοιωτές. Με αυτό τον τρόπο μπορούν να ελεγχθούν τα αυτόνομα συστήματα σε εκατοντάδες παραλλαγές των οδικών και περιβαλλοντικών συνθηκών, ακίνδυνα και παρέχοντας αναπαραγωγιμότητα των σεναρίων δοκιμής.

Ο αριθμός των εργαλείων προσομοίωσης για αυτο-οδηγούμενα οχήματα έχει αυξηθεί τα τελευταία χρόνια, προσφέροντας πληθώρα επιλογών. Χαρακτηριστικά όπως το ανοιχτού κώδικα λογισμικό, η συμβατότητα με πολλές πλατφόρμες, η προσαρμοστικότητα και η τεκμηρίωση είναι ιδιαίτερα σημαντικά για κάθε λειτουργικό περιβάλλον προσομοίωσης, ενώ σημαντικό ρόλο παίζει επίσης και ο τρόπος και η ακρίβεια μεταφοράς του φυσικού κόσμου στο εικονικό περιβάλλον. Κάποιες λύσεις παρέχουν εργαλεία που καλύπτουν όλη τη διαδικασία της ανάπτυξης ενός αυτόνομου οχήματος, ενώ άλλες επικεντρώνονται σε συγκεκριμένα υποσυστήματα ή λειτουργίες, ακόμη και στη μοντελοποίηση ενός μόνο αισθητήρα.

Προσομοιωτές όπως οι AirSim [1], DeepDrive [13], CARLA [16] και ο προσομοιωτής της Udacity [39] είναι ανοιχτού κώδικα εργαλεία, που έχουν υλοποιηθεί, σε ισχυρές μηχανές γραφικών ενώ προσφέρουν την προσομοίωση πλήθους αισθητήρων και APIs σε Python, C, C++ ή Java. Υπάρχουν βέβαια και προσομοιωτές κλειστού κώδικα, όπως το Carcraft [9], που χρησιμοποιείται από τη Waymo για την εκπαίδευση των αυτοκινήτων της σε «ενδιαφέροντα» σενάρια, δηλαδή σε συνθήκες όπου οι έλεγχοι στο φυσικό κόσμο υπέδειξαν την ανάγκη για περισσότερη εξάσκηση.

Το CARLA, που χρησιμοποιήθηκε για τη διεξαγωγή της παρούσας εργασίας, υποστηρίζει την ανάπτυξη, εκπαίδευση και επαλήθευση των αλγορίθμων αυτόνομης οδήγησης με φωτορεαλιστικές προσομοιώσεις. Είναι υλοποιημένο σε γλώσσα C++ ως πρόσθετο της μηχανής γραφικών Unreal Engine [41]. Παρέχει εικονικά αστικά περιβάλλοντα με κτίρια, οχήματα, πεζούς και σηματοδότες, προσαρμόσιμους αισθητήρες και ποικιλία περιβαλλοντικών συνθηκών. Στους αισθητήρες που παρέχονται συμπεριλαμβάνονται LiDARs, GPS αισθητήρες και κάμερες. Ενώ ο προσομοιωτής ενσωματώνει στο backend όλη τη λογική ελέγχου, την απόδοση, τις φυσικές ιδιότητες των οχημάτων και των πεζών, παρέχεται επιπλέον και ένα Python API που επιτρέπει τον έλεγχο οχημάτων, περιβαλλοντικών συνθηκών και αισθητήρων. Το CARLA περιλαμβάνει περισσότερα από 7 διαφορετικά

εικονικά αστικά περιβάλλοντα και δίνει, επίσης, τη δυνατότητα για την υλοποίηση ενός νέου χάρτη επιτρέποντας την ενσωμάτωση των ήδη υλοποιημένων γραφικών στοιχείων και την προσθήκη νέων. Επίσης, τόσο οι δυνατότητες παραμετροποίησης των οδικών συνθηκών και πρακτόρων όσο και η ευελεξία του API που παρέχεται, καθιστούν εύκολη την ενσωμάτωση των τεχνολογικών εξελίξεων, χρησιμοποιώντας αντίστοιχες πλατφόρμες λογισμικού. Μία τέτοια πλατφόρμα που προσφέρει μονάδες για πρόβλεψη και σχεδιασμό με συστήματα κανόνων και νευρωνικά δίκτυα, έλεγχο, εντοπισμό και ανίχνευση με χρήση αισθητήρων στα αυτο-οδηγούμενα περιβάλλοντα είναι το Autoware.AI [5]. Ένα άλλο παράδειγμα είναι το Scenic [18], μία γλώσσα προγραμματισμού για την περιγραφή σεναρίων ως κατανομές συγκεκριμένων διαμορφώσεων των φυσικών αντικειμένων και των πρακτόρων (για παράδειγμα των αυτοκινήτων). Και τα δύο αυτά εργαλεία είναι συμβατά με το CARLA.

Τα συστήματα αυτόνομης οδήγησης, όντας κυβερνο-φυσικά συστήματα, περιλαμβάνουν αλληλεπιδράσεις μεταξύ υπολογιστικών πυρήνων, δικτύων επικοινωνίας και φυσικών περιβαλλόντων. Αυτά τα συστήματα είναι εγγενώς περίπλοκα και μη ντετερμινιστικά. Καθότι λάθη στο σχεδιασμό των αυτο-οδηγούμενων οχημάτων θα μπορούσαν ακόμη και να στοιχίσουν ανθρώπινες ζωές, είναι υψίστης σημασίας η εγγύηση της ορθότητάς τους. Στο πλαίσιο αυτό, οι παραδοσιακές τεχνικές επαλήθευσης (formal verification) δεν αποτελούν μία πρακτική λύση, καθώς η ραγδαία ανάπτυξη του χώρου οδηγεί σε ολοένα και πιο σύνθετα και ετερογενή συστήματα. Επίσης, η λύση του testing αν και χρησιμοποιείται ευρέως για την ανάπτυξη των συστημάτων, δεν αρκεί από μόνη της για να διασφαλιστεί η ορθότητα, αφού τα σενάρια που μπορεί να συναντήσει ένα αυτόνομο όχημα είναι άπειρα.

Το runtime monitoring (επιτήρηση σε χρόνο εκτέλεσης) παρέχει έναν πρακτικό τρόπο επαλήθευσης των κρίσιμων ιδιοτήτων των συστημάτων αυτόνομης οδήγησης σε πραγματικό χρόνο. Ειδικότερα, ένας επιτηρητής χρόνου εκτέλεσης (runtime monitor) παρακολουθεί συνεχώς τη συμπεριφορά του συστήματος και ελέγχει αν συμβαδίζει με μία υψηλού επιπέδου περιγραφή απαιτήσεων, παρέχοντας μία ποσοτική ή ποιοτική ετυμηγορία. Ο καθορισμός των απαιτήσεων γίνεται συνήθως με τη μορφή τύπων χρονικής λογικής (temporal logic). Η τεχνική της επιτήρησης σε χρόνο εκτέλεσης δίνει τη δυνατότητα επικύρωσης αυτόνομων συστημάτων οδήγησης σε ρεαλιστικά σενάρια οδήγησης μέσα σε δυναμικά αστικά περιβάλλοντα. Επίσης, επιτρέπει τη συστηματική εξερεύνηση του πεδίου ορισμού των παραμέτρων του συστήματος.

Παρά το έντονο επιστημονικό ενδιαφέρον για τα αυτόνομα συστήματα σε προσομοιωτές οδήγησης, η έρευνα των τεχνικών επαλήθευσής τους δεν έχει προχωρήσει αρκετά. Πολλοί από τους προσομοιωτές επιτρέπουν την εξαγωγή σημάτων για περαιτέρω επεξεργασία και ανάλυση. Αυτό δεν είναι επαρκές σε πρακτικό επίπεδο, αλλά μπορεί να αποτελέσει είσοδο για έναν επιτηρητή χρόνου εκτέλεσης, που να μπορεί συνεχώς να ελέγχει και να επιβεβαιώνει την ευρωστία των συστημάτων.

Ο πρώτος στόχος αυτής της εργασίας ήταν να διερευνηθούν οι δυνατότητες του CARLA, και κυρίως όσες σχετίζονται με τον έλεγχο των οχημάτων, την παραγωγή και εξαγωγή των δεδομένων της προσομοίωσης και την επαλήθευση των αυτόνομων συστημάτων και αλγορίθμων. Πιο συγκεκριμένα, διερευνήσαμε τη λειτουργικότητα του Python API και των αισθητήρων, που καθιστούν δυνατό τον έλεγχο των αυτοκινήτων και την ανίχνευση του περιβάλλοντος. Εξετάσαμε, επίσης, τις επιλογές που παρέχονται αναφορικά με την επικοινωνία μεταξύ ενός πελάτη και του προσομοιωτή του CARLA. Ο πελάτης μπορεί να συνδεθεί στην προσομοίωση μέσω του Python API, για να κατευθύνει τους πράκτορες ή να λάβει δεδομένα. Τέλος, επικεντρωθήκαμε και στην παραγωγή ποικίλων σεναρίων και την επίτευξη αναπαραγωγιμότητας της εκτέλεσής τους.

Στο πλαίσιο της εξοικείωσης με το CARLA, υλοποιήθηκε ένα Adaptive Cruise Control (ACC) σύστημα. Το σύστημα αυτό όταν ενεργοποιείται σε ένα όχημα, ανιχνεύει με τη χρήση κάποιου κατάλληλου αισθητήρα αν προπορεύεται κάποιο άλλο όχημα σε κοντινή

απόσταση και ρυθμίζει την ταχύτητα του οχήματος που ακολουθεί ώστε να διατηρείται η μεταξύ τους απόσταση ασφαλείας.

Στη συνέχεια, διερευνήσαμε τα μέσα επαλήθευσης των συστημάτων αυτόνομης οδήγησης σε προσομοιωτές, και κυρίως την τεχνική επαλήθευσης σε πραγματικό χρόνο. Ειδικότερα, επεκτείναμε το CARLA ενσωματώνοντας τη βιβλιοθήκη rtamt [30]. Η συγκεκριμένη βιβλιοθήκη υποστηρίζει την ποιοτική και ποσοτική επαλήθευση της ευρωστίας των προδιαγραφών του συστήματος, εκφρασμένων σε Signal Temporal Logic (STL). Με αυτό το εργαλείο, εξετάσαμε τη συμπεριφορά του συστήματος ACC σε δύο σενάρια διαμόρφωσης με διαφορετικά σύνολα παραμέτρων ενός PID ελεγκτή. Επίσης, επαληθεύσαμε την ορθότητα του συστήματος που υλοποιήσαμε, εκφρασμένη με δύο απαιτήσεις απόδοσης. Στόχος μας ήταν η επαλήθευση της ορθότητας του ACC συστήματος σε διάφορα ρεαλιστικά σενάρια οδήγησης σε συνθήκες αστικού περιβάλλοντος.

Contents

Summary **ii**

Περίληψη **iii**

1 Introduction **1**
1.1 Challenges in Autonomous Driving System Design 1
1.2 Urban Driving Simulators and CARLA 2
1.3 Runtime Verification for Autonomous Driving Systems 4
1.4 Thesis Contributions 5

2 CARLA Simulator **7**
2.1 Core Concepts in CARLA 7
2.1.1 Vehicle control 7
2.1.2 Map Waypoints and Route Planning 8
2.1.3 Traffic signs 9
2.1.4 Walker control 10
2.1.5 Sensor suites 10
2.1.6 Weather conditions 11
2.1.7 Simulation recording and replaying 12
2.2 Python API 13
2.2.1 Configuration of the simulation and the actors 13

3 Runtime Verification of CARLA Simulations **16**
3.1 Online Runtime Monitoring in CARLA 16
3.2 STL Runtime Verification for Autonomous Driving Systems 18
3.3 Related Work 19

4 Runtime Verification of an Experimental ACC System **22**
4.1 An experimental ACC system 22
4.2 Design space exploration w.r.t. PID parameters 24
4.3 Requirements for an ACC system 27

5 Conclusions and Future Work **30**

A Installation of CARLA **32**
A.1 Launch the simulator 32
A.2 Build on Linux 32
A.3 Execute python scripts 33

B Signal Temporal Logic (STL) syntax **35**

C Python script fot an rtamt Monitor **37**
C.1 Initialize the monitor with the required specifications: 37
C.2 Online monitoring for three events A, B and C: 37
C.3 Offline monitoring: 38

Bibliography **39**

List of Figures

2.1 CARLA client-server communication [10] 7
2.2 Hierarchy of some of the most important C++ classes 8
2.3 Diversity of cars available in CARLA 8
2.4 Waypoints as vehicle locations 9
2.5 Traffic lights in CARLA 10
2.6 Walker in CARLA 10
2.7 The same scenes with differing weather conditions in CARLA 12
2.8 Server and client side of CARLA 13
2.9 Main simulation loop of autonomous driving control systems in CARLA 14
2.10 Basic classes of the Python API and their relations 15

3.1 Sensing data and simulation tick 16
3.2 Online monitoring CARLA simulations of autonomous driving control systems 17
3.3 STL runtime verification of CARLA simulations with the `rtamt` library 18
3.4 Implementation of `rtamt` monitors for every pair of cars 19

4.1 Modes of the experimental ACC system (a) preceding vehicle out of range, (b) current distance > safety distance (c) current distance < safety distance 23
4.2 PID controller for the ACC system 24
4.3 Robustness of R when the PID is tuned with (a)$K_{P_A}, K_{I_A}, K_{D_A}$ and (b) $K_{P_B}, K_{I_B}, K_{D_B}$ 25
4.4 (a) Distance between host and preceding vehicle and (b) their velocities 26
4.5 Robustness of R when the PID is tuned with $K_{P_A}, K_{I_A}, K_{D_A}$ on a more complex scenario 27
4.6 Robustness of R when the PID is tuned with $K_{P_B}, K_{I_B}, K_{D_B}$ on a more complex scenario 27
4.7 Robustness of requirement (a) R_1 and (b) R_2 28

Chapter 1

Introduction

1.1 Challenges in Autonomous Driving System Design

Although there are still key barriers to overcome in order to achieve full automation, the recent research and the race of the car manufacturers on building autonomous vehicles has made significant progress. Today, some vehicles are semi-autonomous with Advanced Driver Assistance Systems (ADAS) which simplify driving and improve the driver's attention. Adaptive Cruise Control, Anti-lock braking system, Automatic parking and Pedestrian protection system are only a few of these systems aiming to make driving easier and safer.

The challenges posed by most current approaches in the field are centered around lack of safety guarantees and the unleashed costs and risks of testing the run-away technological advances. Ultimately, our desire is to guarantee zero accidents, but this is impossible since multiple agents are typically involved in an accident and one can easily envision situations where an accident occurs solely due to the blame of other agents. In light of this, it is critical to know not only what the minimal requirements that every self-driving car must satisfy are, and how we can verify them but also, under which conditions these requirements are likely to be violated.

To guarantee that a self-driving car is safe and to reduce costs, different scenarios must be modeled and tested. Systematic testing of autonomous vehicles can be performed in simulation or in the physical world. Physical tests offer real testing scenarios, and engineers can use actual vehicles instead of models. However, regulations restrict the use of these vehicles in cities; thus, to perform tests with real autonomous vehicles, one needs access to expensive hardware and field tests, which consume a considerable amount of time.

In addition to that, real-world road tests don't provide realistic validation of autonomous systems, because no two tests can replicate the same conditions ([36]). In the real world, no two situations are exactly equivalent; everything from pedestrian behavior to traffic and weather conditions varies from one moment to the next. Relying on real-world road testing alone is simply unrealistic, because it would take hundreds of years for vehicles to be trained on, not just every road, but on every relevant variable that could affect safe performance on those roads ([4]).

In this context, simulation, modeling and testing has the potential to fill the gap and enable rigorous, controlled, and timely evaluation of autonomous driving systems. These virtual test drives simultaneously test a car's systems against thousands of variations in road conditions, without any real-world safety risks and at the same time, incorporate reproducibility of the verification scenarios.

1.2 Urban Driving Simulators and CARLA

The number of tools for the simulation of autonomous vehicles has increased in recent years, offering a variety of options for selecting the best tool for a specific development. Features like open-source, multi-platform, personalization and documentation are desirable in all simulators. Furthermore, regarding the simulation of such robotic systems, one must take into account all the aspects of physical implementation to further simplify the transition from virtual- to real-world scenarios. Some of the solutions provide a series of toolchains to address the end-to-end process of an autonomous vehicle; others only contemplate some of the subsystems or functionalities (Advanced Driver Assistance Systems (ADAS), Simultaneous Localization And Mapping (SLAM), Driving, traffic, etc.).

Most modern autonomous driving simulators provide fast prototyping, physics engines for realistic movement, usually Unreal Engine [41] or Unity [40], realistic 3d rendering and are dynamic with scripting in programming languages such as C, C++, Python, Java, etc.

AirSim [1] focuses on experimenting with deep learning, computer vision and reinforcement learning algorithms for autonomous vehicles. It extends its functionality by supporting wheeled robotics, aerial drones, and static IoT devices, thus allowing the user to explore their interaction with the self-driving cars. AirSim exposes APIs in Python, C++, C# and Java that support data retrieval from the offered sensors (cameras, GPS, IMU and LiDAR) and the control of vehicles in a platform independent way. To ease the generation of training data it provides a recording mode, which saves pose and images for each frame of the simulation and also a computer vision mode that allows to save images by positioning cameras or simply wandering around the environment using the keyboard. AirSim is open-source and built as an Unreal Engine plugin, which makes it easy to be modified and/or extended. Recently an experimental Unity release has also been developed.

DeepDrive [13] is another open-source simulator, built on Unreal Engine and focused on deep reinforcement learning. DeepDrive supports OpenAI, Baselines PPO2, online leaderboards and it can easily be used with TensorFlow. Its Python API enables control of in-game agents that overtake, follow, route, and negotiate intersections intelligently in one of its three diverse maps. Training data can be collected from GPS, LiDAR, Radar sensors and cameras.

The online education company Udacity [39] built its own simulator for educational purposes, and has now made it available via open-source license. The simulator is built on Unity and offers a training mode that allows to drive a car to generate training data, and an autonomous mode used for testing the machine learning model that drives the vehicles. It offers two driving scenes and GPS, IMU, LiDAR and infrared sensors.

Waymo, that began as the Google Self-Driving Car Project, is using the Carcraft [9] simulator as a testing ground. Originally developed as a way to "play back" scenes that the cars experienced while driving on public roads, Carcraft has taken on an ever-larger role within the self-driving program. Carcraft might simulate driving down a particularly tricky road hundreds of thousands of times in a single day. And crucially, the virtual miles focus on "interesting" miles. These are not boring highway commuter miles but scenarios in which the cars might learn something new. These situations where extra practice is needed are shown by the miles driven on regular roads. Carcraft, which use remains restricted, has an outsized rile in the effort of Waymo to deliver fully autonomous vehicles in the next few years.

The LGSVL Simulator [23] facilitates testing and development of autonomous driving software systems by enabling the simulation of billions of miles and arbitrary edge case scenarios. The simulator is integrated with the open source platforms Apollo and Autoware and built on Unity. LGSVL provides real-time outputs from sensors including camera, LiDAR, RADAR, GPS, and IMU. Environmental parameters can also be changed, including map, weather, traffic, and pedestrians. Python API - scripting functionality is provided for creating scenarios and running automated test cases. The LGSVL simulator is developed by the Advanced Platform Lab at the LG Electronics America R&D Center.

CARLA is an open source photorealistic simulator [16] developed to support the development, training and validation of autonomous driving algorithms. It is written in C++ as a plugin of Unreal Engine. It provides digital assets (urban layouts, buildings, vehicles) and flexible specification of sensor suites and environmental conditions. The available sensors include LiDARs, GPS sensors and cameras. In addition to CARLA Simulator, that embeds all the control logic, the rendering, the physics and the actor properties, CARLA offers a Python API module which enables control of the actors, the environmental conditions and the sensors. CARLA provides more than 7 diverse urban simulated environments, with pedestrians, traffic lights, speed limits, and realistic layout and allows map generation using the already created open assets or by adding more. The community that develops CARLA is really active providing a new version around every two months, also, a detailed documentation is offered for the use of Python API as well as the core concepts of the simulator.

All the aforementioned simulators are focused on autonomous driving, but there are also others that are restricted to specific components of autonomous systems, such as the LiDAR (GLIDAR [43], Helios [7], SIMLidar [28]), the RADAR (RADSim [32]) or the Ultrasonic sensor (SIMSonic [35]). A review of Perception System and Simulators for Autonomous Vehicles presenting most of these simulators was presented in [33].

In order to apply the latest research findings on the simulated cars it is often crucial to combine the simulator used with software platforms. One of the most widely used open-source software for autonomous driving technology is Autoware.AI [5]. It contains modules for localization, detection, prediction, planning and control. Localization is achieved by 3D maps and SLAM algorithms in combination with GNSS and IMU sensors. Detection uses cameras and LiDARs with sensor fusion algorithms and deep neural networks. Prediction and Planning are based on probabilistic robotics and rule-based systems, partly using deep neural networks as well. The output of Autoware is a twist of velocity and angular velocity by which the vehicle is controlled.

Another example is Scenic [18], a domain-specific probabilistic programming language for describing scenarios: distributions over configurations of physical objects and agents (such as vehicles). In particular, Scenic allows the user to construct objects, impose constraints and geometric relationships, as well as assign default values to the properties of objects. Also, Scenic provides an easy way to generalize a concrete scene by automatically adding noise. It can be used to evaluate the accuracy of an ML system under particular conditions, improve performance in edge cases by emphasizing them during training and debug a known failure case by generalizing it in many directions.

Both of these tools can be used with the CARLA simulator. The sensor data from CARLA can be extracted through a python script and passed as an input to the Autoware algorithm, which will then compute the target velocity and angular velocity

for the ego car at the next simulation step. Then, a CARLA client will use this output to calculate the target throttle, brake and steering values, the three parameters of the vehicle's controller. This transition could be done by tuning a Proportional-Integral-Derivative controller (PID). Such a controller was used for this thesis as well and is described in section 4.1.

Using Scenic with CARLA requires only two steps: (1) writing a small Scenic library defining the supported types of objects and the geometry of the workspace; (2) writing an interface layer converting the configurations output by Scenic into CARLA's input format. Step (1) can take advantage of the `Blueprint` library provided by CARLA's Python API. This library (see section 2.2) contains all the supported actors and their attributes and also, allows filtering with regards to specific actor descriptions. The client scripts can additionally retrieve information for the map's geometry by simply saving them in a file or by loading them in a tuple format (see section 2.1.2). For step (2), the output of Scenic will be used to configure the scenario to be executed, by accordingly spawning the actors, assigning the right properties and configuring the environmental conditions.

1.3 Runtime Verification for Autonomous Driving Systems

Autonomous Driving Systems are Cyber-Physical Systems (CPS) and thus, involve deep interactions between computation cores, communication networks, and physical environments. These systems are inherently complex and highly nondeterministic. Since errors in the systems of a self-driving car used in real-life could mean the difference between life and death, it is significantly crucial to guarantee correctness. While being one of the, if not the, most important problems in designing complex autonomous driving systems, this task is also rather challenging. To this end, traditional formal verification techniques (such as model checking [12]) and testing can be used and are indeed powerful approaches for improving the systems' dependability. However, formal verification becomes impractical to completely verify these systems as development in autonomous driving leads to more and more complex and heterogeneous systems causing the state explosion problem. Testing is widely used during the development of such systems but is insufficient if applied alone to ensure correctness since the scenarios a self-driving car can possibly come across are limitless, thus leaving space for undetected errors. Therefore, driving in uncertain environments that have not been tested can result in the loss of property or even human lives.

Runtime monitoring [22, 19, 27], considered as a lightweight verification technique, provides a practical way to monitor and verify critical properties of CPS systems at runtime. In this context, a runtime monitor continuously observes the behavior of the system and detects if it is consistent with a given high level specification, providing a qualitative or quantitative verdict. The specification of the requirements is usually given as a temporal logic formula, e.g., linear temporal logic (LTL), computation tree logic (CTL) and signal temporal logic (STL), and is then used to synthesize a runtime monitor by automatic tools. Runtime monitoring has been used widely as a means to increase the dependability of software systems.

Recent advances in runtime verification render it a promising perspective for a multitude of reasons. First, reachability analysis of embedded control systems is characterized by well-known theoretical limitations [42], which practically exclude the possibility of being applicable in real-scale industrial problems [20]. Second, runtime verification of simulation traces allows for validating systems in realistic

use case scenarios with complex dynamics. Third, the recent advances on runtime verification of STL properties [25] enable the systematic design space exploration (e.g. through property falsification and parameter synthesis) [6].

This new landscape of analysis capabilities also opens prospects for filling the gap in providing adequate means for qualitative and quantitative evaluation of the simulated models.

Regarding the challenge of validating autonomous driving systems, though there is a high interest to further development of simulators towards providing a richer set of environment models, driving scenarios and novel machine learning use cases, little has been done for the exploration of techniques for the verification of the simulated systems. Most of the simulators, in their basic configuration, allow to export simulation traces that will have to be further post-processed. This alone is not sufficient to effectively validate autonomous driving systems at their early design stage. The traces can be used as input signals for a runtime monitor, in order to test and verify the robustness of autonomous driving systems with machine learning components in realistic urban driving scenarios.

1.4 Thesis Contributions

This first goal of this thesis was to explore the CARLA simulator and its functionalities with a main focus on controlling the vehicles, generating and extracting data from the simulated scenarios and testing the autonomous systems or algorithms. To control the cars and sense the environment, we experimented with the capabilities of the Python API and the implemented sensors. Achieving precision in applying the vehicle control and relating the data from the various sensors, urged the exploration of the supported options for the client-server synchronization. Finally, we also focused on generating diverse scenarios and achieving reproducibility. In order to experiment with these functionalities an Adaptive Cruise Control system was implemented.

Secondly, we worked towards the verification of autonomous systems implemented in simulators, CARLA in our case, and especially their runtime verification ensuring safety or performance requirements expressed in Signal Temporal Logic. More specifically, we have extended CARLA by integrating the `rtamt` library [30], for the online runtime verification of STL properties. `rtamt` supports the qualitative and quantitative verification (property robustness) of STL specifications. This new tool combination allowed (i) the design space exploration of the Adaptive Cruise Control system, in two variants, with respect to various PID control parameters and (ii) the quantitative verification of performance requirements that are necessary for any ACC system [37]. Our experimentation with the CARLA model took place in the context of a set of diversified scenarios, which guarantee that the ACC system is effectively validated under realistic urban driving conditions.

In next Chapter, the basics of the CARLA simulation environment and a description of its basic functionalities, regarding the server and the client side, are presented. Chapter 3 refers to the integration of CARLA with the `rtamt` library for online runtime monitoring as well as to the syntax of STL and its semantics. In the same chapter the relevant research is reviewed and related to the proposed approach of this thesis. Chapter 4 presents the experimental results from the runtime verification of our ACC system. Finally, in the last section we summarize our achievements and discuss interesting future research prospects.

In the Appendix A, we provide the required steps for installing, building, executing and updating CARLA in a Linux environment. Appendix B refers to the syntax of STL, to aid the reader understand the formulas mentioned in this thesis. In Appendix C, we present an example python script, which incorporates an `rtamt` monitor in CARLA.

Chapter 2

CARLA Simulator

CARLA (Car Learning to Act) [16] is an open source simulator for urban driving built on top of the Unreal Engine game engine. Uniquely, the content of urban environments provided with CARLA is also free. It includes urban layouts, a multitude of vehicle models, buildings, pedestrians, street signs, etc. The simulation platform supports flexible setup of sensor suites and provides signals that can be used to train driving strategies, such as GPS coordinates, speed, acceleration, and detailed data on collisions and other infractions. A wide range of environmental conditions can be specified, including weather and time of day.

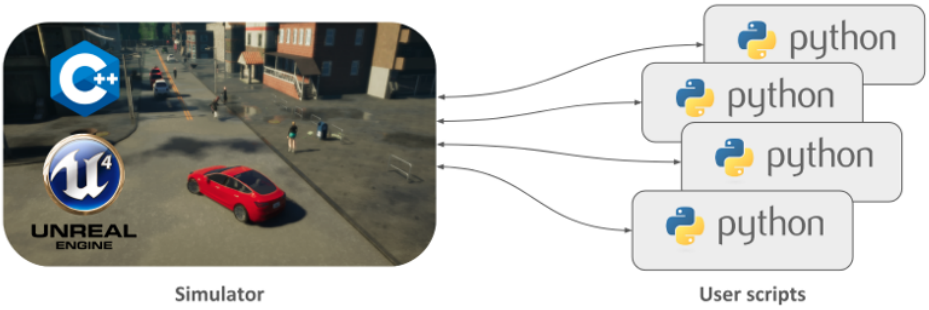


FIGURE 2.1: CARLA client-server communication [10]

A simulation is composed of: (i) the CARLA Simulator that computes the physics and renders the scene and all actor properties, (ii) client scripts written using a Python API, that allows control of the actors, sensors and environmental conditions (see Figure 2.1).

2.1 Core Concepts in CARLA

This section presents the core concepts of CARLA and how these are implemented on the server side of the simulator and accessed by the Python API. We focus mostly on the vehicles' behavior and control. The back-end of the simulator is written as a plugin of Unreal Engine in C++. The documentation of the source code can be found at the C++ reference of CARLA [8]. The hierarchy of some of the most important classes, with regard to the concepts described in this section, of the CARLA backend are presented in Figure 2.2.

2.1.1 Vehicle control

The server receives instructions from the clients regarding the spawning of a vehicle at a specified location, its properties and control messages. Spawning is arranged by

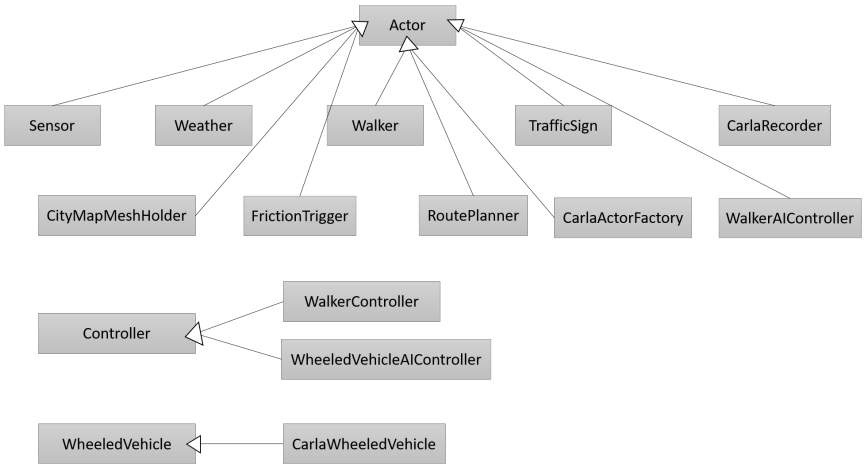


FIGURE 2.2: Hierarchy of some of the most important C++ classes

the `CarlaActorFactory` class based on a description for the actor and a map location. In CARLA, a vehicle is controlled by applying the desired throttle, brake, angle, handbrake and reverse speed (class `CarlaWheeledVehicle`). While on self-driving mode, CARLA assigns a list of destinations to each car via the AI Controller (class `WheeledVehicleAIController`). The destinations are points on the map, that can be reached by a car. In every tick of the clock inside the simulator, the steering value is calculated according to the next destination. After that, CARLA calculates the speed. The car stops if a red traffic light or an obstacle is detected, otherwise it continues driving to its set destination. If its current speed is less than the speed limit it accelerates, otherwise its speed remains equal to the speed limit of the street.

CARLA offers simulated models of many real-world cars such as the Ford Mustang, Nissan Micra, Seat Leon, Toyota Prius and many more, as well as motorcycles and even bikes. Additionally one can add a new vehicle by using the general skeleton provided by CARLA and selecting the desired physical assets.

FIGURE 2.3: Diversity of cars available in CARLA

2.1.2 Map Waypoints and Route Planning

In CARLA, the location of a vehicle must be on one of the waypoints, in every tick of the clock. These waypoints are positions on the map on every possible vehicle location. For every lane, these points have a certain distance between them. For each waypoint, CARLA provides several information such as transform (position and orientation of the waypoint according to its lane), road id and lane information through the Python API. The lane information, consists of the lane id, width, change

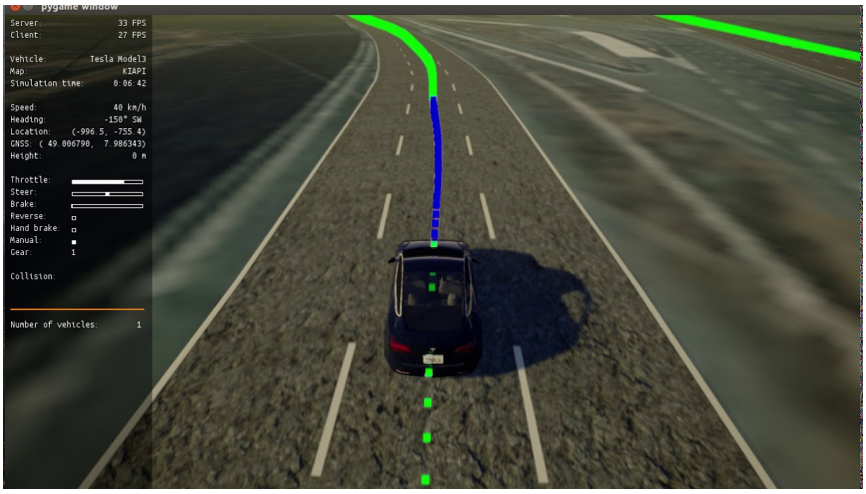


FIGURE 2.4: Waypoints as vehicle locations

(whether it is allowed to turn left and/or right), type (parking, rail, sidewalk, stop, biking, etc.), right and left lane marking type (solid, broken, etc.). Additionally, the Python API includes methods that return the next or previous waypoints (the waypoints that are in an approximate distance from the current one in the same or opposite direction of the lane). In Figure 2.4, the waypoints are shown in green and blue color. If two waypoints are closer than a threshold they are considered as one by CARLA.

While in autopilot mode, for every vehicle and walker CARLA generates a list of random locations and selects one of them as their first destination. Every time an actor reaches its destination CARLA randomly selects another one from the list or generates new locations if the list is empty. While moving, if there is only one waypoint ahead of the actor then the actor is moved to that waypoint. If there are more waypoints ahead -which means that the actor has to choose between different lanes or roads- then some route planning algorithms are used to choose the appropriate one.

The topology of the simulator's map is represented by a graph using the OpenDRIVE format [31]. A minimal graph of the topology can be retrieved from the clients (using the class `Map`). The graph is stored in tuples containing pairs of waypoints located either at the point a road begins or ends. The first one is the origin and the second one represents another road end that can be reached. This graph can be loaded into NetworkX [29] to work with. Output while saving the graph from the Python API could look like this: [(w0, w1), (w0, w2), (w1, w3), (w2, w3), (w0, w4)] or be saved in an OpenDRIVE file.

2.1.3 Traffic signs

The traffic signs in CARLA are either speed limit signs or traffic lights. Although in the backend there are instances of traffic signs according to which the vehicles get the speed limit, these are not presented in the graphical city of CARLA. For the traffic lights, the time for each state (red, green, yellow) can be set. Each traffic sign is situated inside a "box" in the map. When a traffic light changes its state it casts all the vehicles that are inside a limit box near its location and notifies them for the current state (see Figure 2.5).

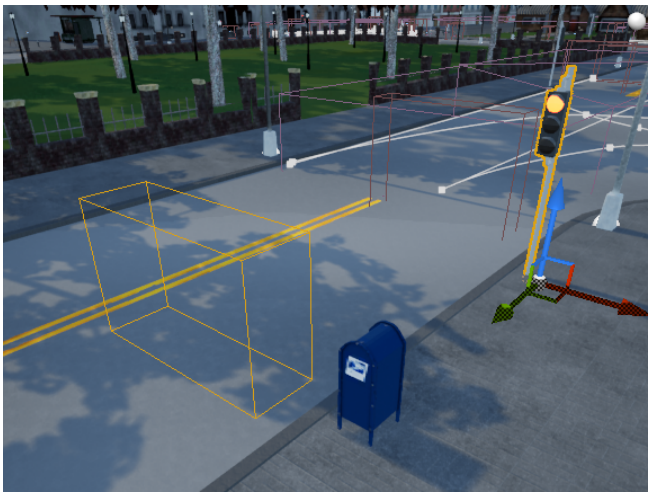


FIGURE 2.5: Traffic lights in CARLA

2.1.4 Walker control

Spawning a pedestrian is instructed by the client with a description of its properties and a location, the same way vehicles are spawned as well. CARLA calculates the speed of the walkers, that should not exceed a maximum limit, checks when they need to stop/slow down and assigns the destinations similarly to vehicles as descibed in section 2.1.2 (using the class `WalkerAIController`). The speed and destination of a walker can be also set by the client app which connects to the server by calling functions of the class `WalkerController`. CARLA provides different models for pedestrians and allows the creation of new ones by providing a customizable skeleton.

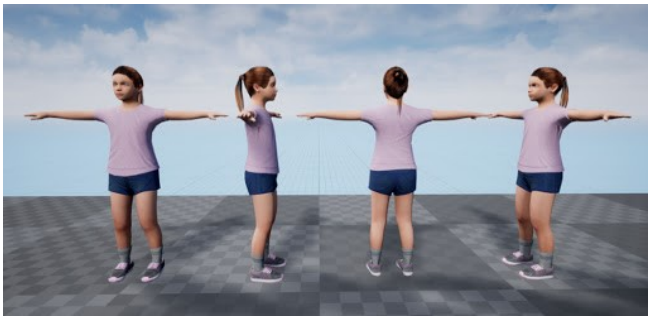


FIGURE 2.6: Walker in CARLA

2.1.5 Sensor suites

CARLA provides a variety of sensors that can be spawned using the Python API with the according specification. In the client script each time a new measurement of a sensor is received a callback is executed. This callback can be set through the `Sensor` class and must accept as an argument the sensor data. These contain a frame count, a timestamp, the sensor's location and other information according to the sensor they refer to.

More specifically, one of the included sensors is the Collision detector, which registers an event each time its parent actor collisions against something in the world. Several collisions may be detected during a single simulation step. To ensure that

collisions with any kind of object are detected, the server creates "fake" actors for elements such as buildings or bushes so the semantic tag can be retrieved to identify it.

Lane invasion detector registers an event each time its parent crosses a lane marking. The sensor uses road data provided by the OpenDRIVE description of the map to determine whether the parent vehicle is invading another lane by considering the space between wheels.

Obstacle detector registers an event every time the parent actor has an obstacle ahead. In order to anticipate obstacles, the sensor creates a capsular shape ahead of the parent vehicle and uses it to check for collisions. From the Python API, the required distance to scan for obstacles can be set, as well as a filter for the target obstacles.

The RSS sensor integrates the C++ Library for Responsibility Sensitive Safety in CARLA. It calculates the RSS state of a vehicle in order to use this data to adapt a `carla.VehicleControl` before applying it to a vehicle. These controllers can be generated by an Automated Driving stack or user input.

Other provided sensors are the GNSS, IMU, LiDAR, RADAR and a variety of cameras, i.e. the depth camera; codifies the distance of each pixel to the camera, the RGB camera; acts as a regular camera by capturing images, the semantic segmentation camera; classifies each object in sight by color displaying, the dynamic vision sensor (DVS); an event camera which measures changes of intensity asynchronously, in the form of a stream of events, which encode per-pixel brightness changes.

Using the sensors from the client side requires three steps. First it is needed to set the desired sensor:

```
# Find the blueprint of the sensor.
blueprint = world.get_blueprint_library().find('sensor.camera.rgb')
# Modify the attributes of the blueprint to set image resolution and field
    of view.
blueprint.set_attribute('image_size_x', '1920')
blueprint.set_attribute('image_size_y', '1080')
# Set the
blueprint.set_attribute('sensor_tick', '1.0') # time in seconds between
    sensor captures
```

Secondly, the sensors have to be spawned. This can be done on an actor, for instance the ego vehicle, or at a point in the map.

```
transform = carla.Transform(carla.Location(x=0.8, z=1.7))
sensor = world.spawn_actor(blueprint, transform, attach_to=ego_vehicle)
```

The last step is specifying a callback function to be called when the sensor generates new data.

```
# save_image() will be called each time a new image is generated by the
    camera.
sensor.listen(lambda data: save_image(data))
```

2.1.6 Weather conditions

In CARLA the weather is set by the client in the form of setting the desired values to the weather parameters, i.e. cloudiness, precipitation, wind and fog intensity, wetness and sun angle. These parameters can be also changed while the simulation is running to simulate the real-life changes in the weather conditions.

```
weather = carla.WeatherParameters(
                cloudiness=80.0,
```

```
                    precipitation=30.0,
                    sun_altitude_angle=70.0)

world.set_weather(weather)
```

CARLA also provides some weather presets that can be directly applied to the world and scripts that change the weather parameters in real-time.

FIGURE 2.7: The same scenes with differing weather conditions in CARLA

Source: technologyreview.com

2.1.7 Simulation recording and replaying

The recording feature allows storing all the events happened during the simulation in the recorder file in order to be able to reenact it later on. This can be used for reproducibility reasons while testing an autonomous system. The binary file storing all the data is written on the server side. However, the recorder is managed from the Python API by a client script. The data are stored in every tick of the simulator clock. For each vehicle these information consist on its position, id, steering, throttle, brake, etc., while for a walker on its position, id and speed. Information are also saved for the traffic lights (id, state) as well as the map, date and time when the simulation was recorded.

You can start the recording using the following command by specifying a file name:

```
client.start_recorder("/home/carla/recording01.log")
```

To stop the recording, at any time from the client script the call is also straightforward:

```
client.stop_recorder()
```

Later on this file can be replayed causing actors that appear in the recording to be either moved or re-spawned to emulate it. Those that do not appear in the recording will continue their way as if nothing happened. CARLA also provides some high-level queries to trace and study the events that appeared in a recorded scenario, such as collisions and blocked actors for a certain amount of time.

2.2 Python API

Most of the aspects of the simulation are accessible from the Python API. With Python scripts it is possible to spawn actors, attach sensors to them and, then, retrieve the sensor data, process them and compute the parameters needed by the controller (throttle, brake and steering). Computed values are then sent back to CARLA Simulator, thus forming an ever-running client-server interaction loop.

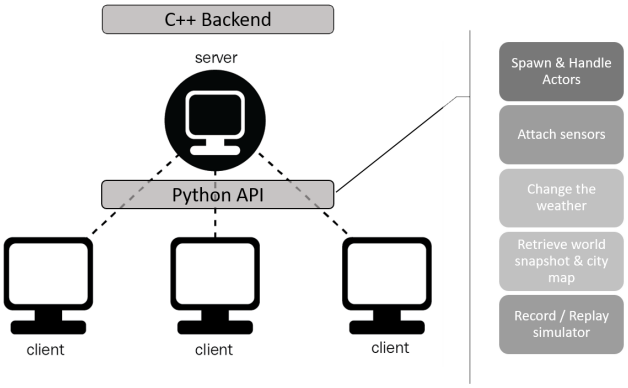


FIGURE 2.8: Server and client side of CARLA

A client script consists of two parts:

- In the first part, connection with the CARLA Simulator is established, sensors are attached to the controlled vehicles and the actors of the simulated scenario are spawned or destroyed. At this point, information on the world can be retrieved, weather parameters can be set, actors can be spawned at specified locations and orientations and sensors can be attached to the vehicles.
- The second part (Figure 2.9) contains the control algorithm and the client-server interaction loop. The client retrieves the state of the actors and the raw data produced by the sensors, to which it reacts by calling the callback function. Then it responds (via the `carla.VehicleControl` object) with the vehicle control signals that are subsequently applied to the `Vehicle` object (method `apply_control()`).

2.2.1 Configuration of the simulation and the actors

The `Client` class is used to set the client and returns an object that can be used to access all the information of the running simulation.

```
# Configure the client
client = carla.Client('localhost', 2000)
client.set_timeout(2.0)
```

After setting the client in order to connect to the server side, the running `World` instance be retrieved. This instance includes methods to get information such as the blueprint library, the simulator's map, the current weather conditions and the spawned actors. Also, it provides methods to set the weather conditions, get recommended destination positions for pedestrians, spawn actors and return the current snapshot of the world, which contains a list of all the spawned actors.

```
# Retrieve the world that is currently running.
world = client.get_world()
```

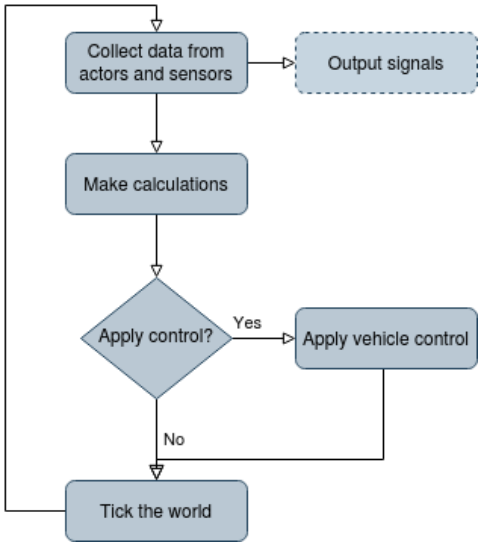


FIGURE 2.9: Main simulation loop of autonomous driving control systems in CARLA

The world object provides access to the `BlueprintLibrary` class. This class contains a list of `ActorBlueprint` elements. Blueprints have an ID to identify them and the actors spawned with it and attributes that can be customized. The library can be read to find a certain ID, choose a blueprint at random, or filter results using a wildcard pattern.

```
blueprint_library = world.get_blueprint_library()
bp = random.choice(blueprint_library.filter('vehicle'))

bp.set_attribute(_attr_name_, _attr_value_)

# Give an initial random transform to the vehicle, from the list of
    recommended spawn points of the map.
transform = random.choice(world.get_map().get_spawn_points())

# Tell the world to spawn the vehicle.
vehicle = world.spawn_actor(bp, transform)

# Enable autopilot mode.
vehicle.set_autopilot(True)

# Destroy the actor.
# If not destroyed the actors remain in the simulation even after quiting
    the Python script.
actor.destroy()
```

At any time during the simulation, the client can retrieve information for a vehicle using the `Vehicle` class. These consist on the current location of the vehicle, the speed limit in the road it is driving, the state of the traffic light affecting the vehicle (if there is one), etc. Applying control to a vehicle can be done by setting various parameters such as throttle, steering, brake, hand brake, etc. Similarly, information could be retrieved for the other actors as well, i.e. pedestrians, sensors and traffic signs.

```
print(vehicle.get_acceleration())
print(vehicle.get_velocity())
```

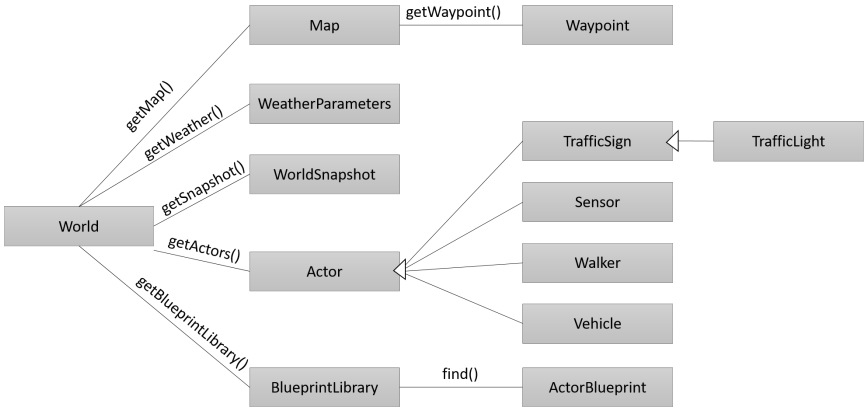


FIGURE 2.10: Basic classes of the Python API and their relations

```
location = vehicle.get_location()
location.z += 10.0
vehicle.set_location(location)
```

```
vehicle.apply_control(carla.VehicleControl(throttle=1.0, steer=-1.0))
```

The hierarchy of some of the basic classes of the Python API and their interactions are shown in Figure 2.10.

Chapter 3

Runtime Verification of CARLA Simulations

3.1 Online Runtime Monitoring in CARLA

In CARLA, the simulated world, which is maintained by the server, has its own clock and time. The time-step can be fixed or variable depending on the user preferences. By default, CARLA runs in asynchronous mode: the server runs the simulation, without waiting for the client, which receives sensor data via a callback function that is called whenever there is new data generated by the sensor(s) or by calling functions of the actors, that return values calculated at the last tick. In synchronous mode, the client script imposes total control over the simulation and its information: the server waits for a client tick (`world.tick()`), before updating to the following simulation step.

Figure 3.1 presents the order in which the control functions and the sensors' listeners are executed as well as when the actors' variables are updated with regards to the simulation tick.

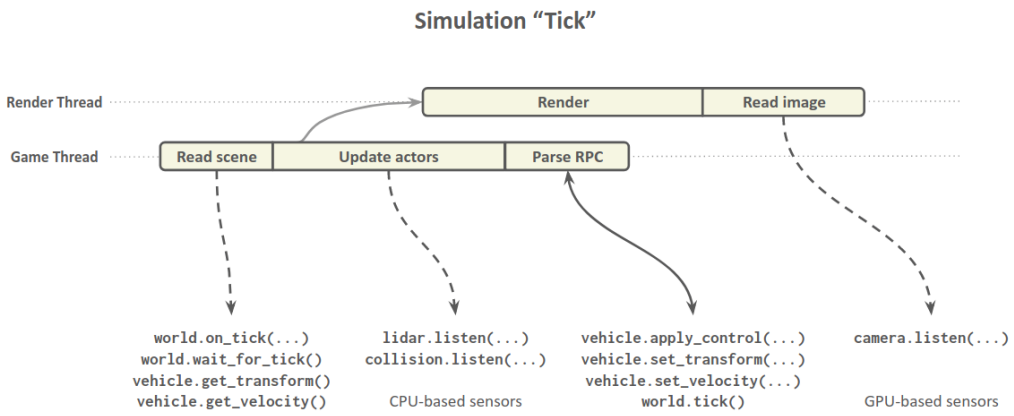


FIGURE 3.1: Sensing data and simulation tick

Source: github.com/carla-simulator/carla/issues/1563

Regarding the computation of physics in the simulated world, the time-step duration matters. When the time progresses, more and more variables are involved, and the simulation may become imprecise. Therefore, physics must be computed within sufficiently small time-steps (not greater than 0.1 sec).

To inspect the behavior of actors (e.g. their location, velocity, acceleration), CARLA provides client functions used to access the values at the latest simulation tick. The

data retrieved in this way is reliable, if the simulation runs in synchronous mode and data is therefore stored in signals (see Figure 2.9).

An example script for storing a signal containing the id, velocity, acceleration and location of each vehicle of the simulation is presented bellow:

```
def vehicle_signal(self):
  timestamp = world.get_snapshot().timestamp
  vehicle_list = world.get_actors().filter('vehicle.*')
  for vehicle in vehicle_list:
    sec = timestamp.elapsed_seconds
    id = vehicle.id
    location = vehicle.get_location()
    velocity = vehicle.get_velocity()
    acceleration = vehicle.get_acceleration()
    signal.append("%d,%d,%d,%d,%d,%f,%f"
                  %(sec, id, l.x, l.y, l.z, v, a))
    return signal
```

For online monitoring, we opted to use synchronous mode with fixed time-step, which ensures that the simulation time and physics will be in synchrony and the sensed signals are reliable (Figure 3.2).

```
settings = world.get_settings()
settings.fixed_delta_seconds = 0.05 # Set a fixed time-step
settings.synchronous_mode = True # Enables synchronous mode
world.apply_settings(settings)

# spawn actors and attach sensors
while True:
world.tick()
# retrieve data and apply controls while the simulation is frozen
```

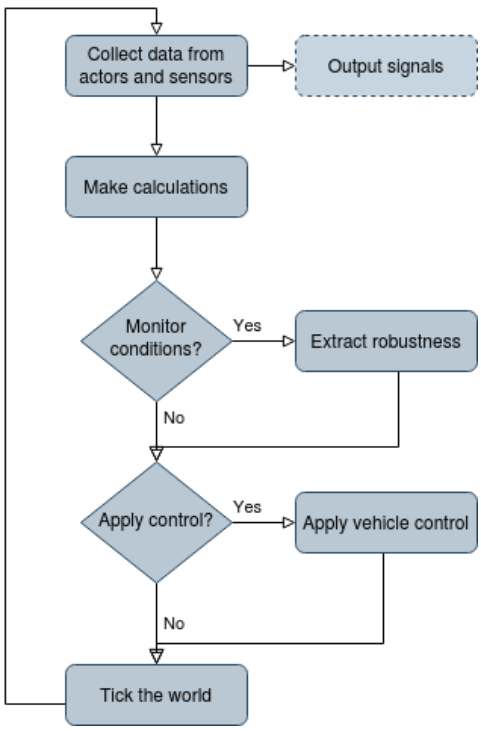


FIGURE 3.2: Online monitoring CARLA simulations of autonomous driving control systems

Online monitoring is naturally restricted to only a single pass through the simulation trace, which poses important challenges for the expressiveness of the specification language and the monitoring algorithm to be used regarding the properties that can be monitored. Moreover, to assess the performance of closed-loop autonomous systems in complex urban driving scenarios, we need to be able to determine the satisfaction/violation of a property specification based on a robustness degree function (see Figure 3.2), i.e. a means to indicate how far is the monitored trace from satisfying or violating a specification. This allows to interpret the performance of the system design under various parameters, scenario events (e.g. street signs) and environment perturbations, as opposed to the binary pass/fail answer of qualitative verification, which is not very informative.

An important consideration is the reproducibility of simulation scenarios, which can be achieved by enabling via the Python API a simulation recorder, the functionality of which was described in section 2.1.7.

3.2 STL Runtime Verification for Autonomous Driving Systems

Autonomous driving systems, like those simulated in CARLA, are cyber-physical systems, i.e. they are physical systems controlled by an embedded computational core. As such, they generate behaviours that are treated as continuous signals and trajectories. To avoid manual inspection of simulation traces or the need for ad-hoc test oracles, we rely on a runtime verification approach based on STL [25, 6], a specification formalism capable to express temporal properties for continuous signals. Appendix B provides background on STL and its syntax.

STL runtime verification of autonomous driving systems takes place through the online monitoring of CARLA simulation traces, as described in the previous Section, using the recently introduced `rtamt` library [30]. `rtamt` generates online robustness monitors from STL specifications. Thus, for any signal in the form of a sequence of (time, value) pairs, `rtamt` computes at different points in time how far is the observed signal from satisfying or violating the STL property.

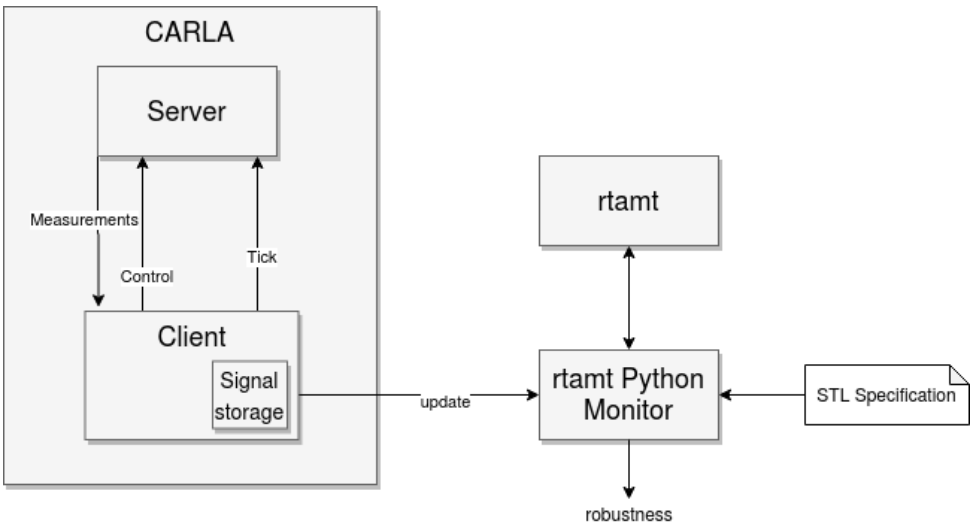


FIGURE 3.3: STL runtime verification of CARLA simulations with the `rtamt` library

`rtamt` accepts as input bounded-future STL, i.e. specifications for which the future temporal operators (eventually, always and until) are restricted to bounded intervals. These specifications are parsed in order to generate, as output, the corresponding monitors. Online monitoring takes place through evaluating the (equivalent) past STL specification, according to its quantitative semantics, interpreted in discrete or dense time [15]. When using discrete-time monitors, sensing of inputs and output generation are done at a periodic rate. On the other hand, dense-time monitors implement an event-driven monitoring approach aiming to compute the min and max of a numeric sequence over a sliding window.

Figure 3.3 shows how an `rtamt` monitor is integrated with a CARLA simulation in synchronous mode and fixed time-step. The (time, value(s)) pairs with data, for every car with the autonomous driving system enabled, are stored by the client at each step. This requires monitoring one or more events, like for example that the host (ego) vehicle is behind another car (see also Figure 3.2). If the monitored conditions hold true, the `rtamt` monitor's `update` function is called to compute the robustness measures from the stored signals. In effect, we have a non-intrusive monitoring solution, since all computations take place, while the simulated time is frozen.

An example python script which integrates a simple `rtamt` monitor is presented on the Appendix C. In our approach, we used one `Monitor` instance for each pair of a preceding and a succeeding car. To do so, we implemented a `MonitorsController` which receives sensor information from every `ObstacleDetector` sensor through the callback functions and directs these data to the according monitor. Also, in every tick when the simulation time is frozen it orders every Monitor to update its robustness according to the new data. The controller stores and updates a dictionary (`follow`) with an entry for each car that has its ACC enabled and is currently following another vehicle. For each entry the dictionary contains a pair of `{preceding vehicle, monitor}` as its value. We also note that the `monitor` class can check for the evaluation of one or more conditions at the same time.

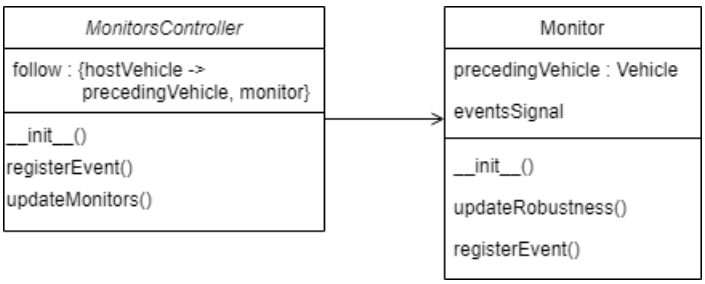


FIGURE 3.4: Implementation of `rtamt` monitors for every pair of cars

3.3 Related Work

We review related work on simulation-based validation of cyber-physical systems guided by formal specifications, as well as tools and methods for property falsification, parameter synthesis and requirements-driven system testing. We are interested in particular for CARLA-related tools, e.g. for scenario generation and for STL patterns focusing on the sort of systems to which our approach refers.

In [17], the VERIFAI toolkit was introduced, for the formal design and analysis of systems that include artificial intelligence and machine learning components. VERIFAI analyzes traces generated from a simulator for the domain of interest, which

may be CARLA since the authors report that they have interfaced their tool with it, for simulating realistic urban driving scenarios. No results are presented in [17] with respect to this aspect, but the VERIFAI approach presents some commonalities with our work, as well as many interesting opportunities for future research. First, VERIFAI has a much wider scope, since it aims to address challenges like the absence of specifications for the perception components of cyber-physical systems, the analysis (and design) of machine learning components and the modeling of the environments (e.g. distribution assumptions made by machine learning components).

In comparison with our work, the most interesting aspect of VERIFAI is that it works through offline monitoring system-level properties in Metric Temporal Logic (MTL) [2], while supporting both its Boolean and quantitative semantics. Monitors can be used to output falsifying traces, as well as a feedback to direct sampling to find falsifying scenarios. Similar analysis is also possible in our case, since `rtamt` can be easily used for offline monitoring and CARLA supports the output of simulation traces, as we already noted in Section 3.1. MTL, as opposed to STL, is interpreted exclusively in continuous-time and does not support atomic propositions, i.e. MTL specifications only use predicates over real-valued variables. STL integrates smoothly the propositional context of Linear Temporal Logic with the possibility of using predicates over real-valued variables, whereas `rtamt` can also generate dense-time monitors, as noted in Section 3.2.

Automotive control systems are characterised by performance requirements that typically arise out of test scenarios, which in our case are the urban driving scenarios supported by CARLA. This is in contrast with other systems, where formal requirements may be introduced during the initial requirements engineering phase and they do not evolve in the course of the system development cycle. Automotive control systems exhibit complex behaviors that are difficult to anticipate at design time. However, formal verification can also play a constructive role: through simulation-based analysis we can identify behaviours that can be subsequently captured as new requirements to be formalised and included in the system requirements of the next development cycle.

This perspective is proposed in [21], where a library of signal templates is presented that can be reused, for specifying STL automotive control requirements. The original motivation of that work is the difficulty in specifying meaningful requirements, for automotive control systems. As the authors note, there is often a mismatch between formal specification languages, such as STL, and the kinds of behaviours considered by control engineers. Therefore, that work introduces a set of behaviour classes that control engineers typically want to avoid (ringing, excessive overshoot, slow response time, steady state error etc.), such that it will be easier to specify STL requirements stating that these behaviours are excluded. This kind of requirements is probably easier to be checked over simulation traces produced by a Simulink model of the system under design, whereas for the requirements (adopted from [37]) of our ACC system in Chapter 4, we advocate their validation over realistic driving scenarios generated by CARLA.

Worth to mention are also the S-TaLiRo [3] and Breach [14] tools, which support the sensitivity analysis and falsification testing over simulation traces generated from Simulink models. Sensitivity of the model robustness to its STL requirements takes place by uniformly varying the input parameters of the simulated model, whereas falsification testing aims to find an input signal that violates a requirement. Another interesting prospect is the approach presented in [38] for requirements-driven system testing. However, it is still not clear how these analyses can be applied in the context of CARLA simulations. There is only one research work, known

to us, towards the systematic testing of automotive driving systems in real-world scenarios [24], but the possibility to integrate it with CARLA has not yet been researched.

Chapter 4

Runtime Verification of an Experimental ACC System

4.1 An experimental ACC system

Adaptive Cruise Control (ACC) systems extend the functionality of conventional cruise control (CCC) systems by having the capability to adjust the host vehicle's velocity and assure a safety distance to the preceding vehicle through controlling its throttle and/or brake. A key part of an ACC system is the range sensor, which is used to measure the distance from the front vehicle. In CARLA, we use the obstacle detector sensor that detects obstacles, including vehicles, located within a specified distance from the host vehicle, towards its traveling direction.

Our experimental ACC system works in three modes, as depicted in Figure 4.1. In mode (a) the ACC system is disabled since there is no preceding vehicle or the preceding vehicle is out of range. Therefore in this mode, the velocity of the host vehicle is controlled by the CARLA server. The ACC system of the host vehicle is enabled when the preceding vehicle is too close (distance below a fixed threshold r) or if it is moving slowly. In this case, if the current distance between the two cars is greater than the safety distance the system enters mode (b). The ACC system controls the throttle and the brake, in order to keep the distance higher or equal to the safety distance, which is dynamically calculated, as specified by Definition 4.1.1. Finally, there is an emergency mode (c) in which the distance of the vehicle is lower than the safety distance and, thus, the host vehicle decelerates until the systems enters a safe mode again. This mode can be enabled in situations that the preceding vehicle reduces its speed abruptly to prevent an accident until the PID controller adjusts again (transition from mode (b) to mode (c)) or in situations where a vehicle suddenly enters the road in front of the host vehicle (transition from mode (a) to mode (c)).

Definition 4.1.1. [34] **(Safety Distance)** A longitudinal distance between a car c_h that drives behind another car c_p, where both cars are driving at the same direction, is safe w.r.t. a response time ρ, if for any braking of at most $a_{brake_{max}}$, performed by c_p, if c_h will accelerate by at most $a_{accel_{max}}$ during the response time, and from there on will brake by at least $a_{brake_{min}}$ until a full stop, then it won't collide with c_p.

Let v_h, v_p be the longitudinal velocities of the cars. Then, the safe longitudinal distance between c_h and c_p is:

$$SD = \left[v_h \cdot \rho + \frac{1}{2} a_{accel_{max}} \cdot \rho^2 + \frac{(v_h + \rho \cdot a_{accel_{max}})^2}{2 \cdot a_{brake_{min}}} - \frac{v_p^2}{2 \cdot a_{brake_{max}}} \right]_+$$

where $[x]_+ := max\{x, 0\}$.

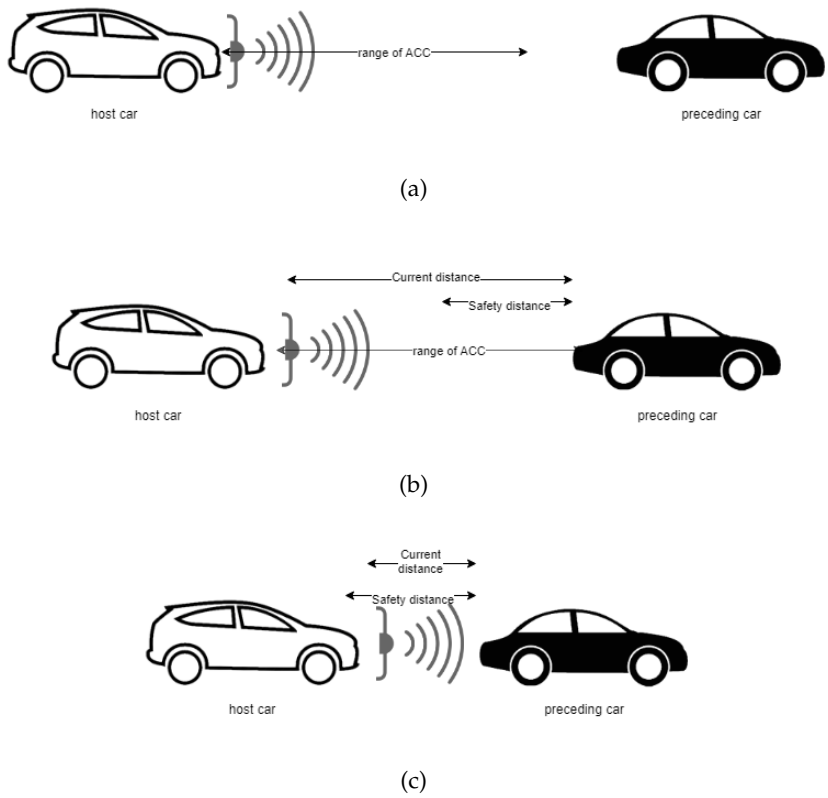


FIGURE 4.1: Modes of the experimental ACC system
(a) preceding vehicle out of range,
(b) current distance > safety distance
(c) current distance < safety distance

In our case, the response time ρ is equal to the time elapsed between two simulation steps, and since the simulation runs in synchronous mode this is fixed and set to 0.05 seconds.

For $a_{accel_{max}}, a_{brake_{min}}$ and $a_{brake_{max}}$ we parameterized the simulation model using indicative values from vehicle specifications by car manufacturers. The values used are: $a_{accel_{max}} = 5.4\, m/sec^2, a_{brake_{min}} = 2.9\, m/sec^2, a_{brake_{max}} = 9.8\, m/sec^2$.

The ultimate aim of the ACC is to allow the host vehicle traveling with a velocity v_h at most equal to v_p, within the range of the road speed limit (assigned by CARLA), while minimizing the distance d from the preceding vehicle, without violating SD.

The available time, before the two vehicles get closer than SD is:

$$time_{safe} = \frac{d - SD}{v_h - v_p}$$

Thus, the target velocity v_{tar} and target acceleration a_{tar} for the host vehicle to have in next simulation sample (time-step) are:

$$a_{tar} = \frac{min\{v_p, speed_{limit}\} - v_h}{time_{safe}}$$

$$v_{tar} = v_h + \rho \cdot a_{tar}$$

The ACC system function is driven by the proportional-integral-derivative controller

(PID) shown in Figure 4.2, which computes the needed throttle and brake based on v_{tar} and the actual velocity (v_h). The PID controller continuously calculates an error value $e(t)$, as the difference between the desired set point (in our case v_{tar}) and a measured process variable (v_h), and applies a correction based on three terms, namely the proportional, the integral, and the derivative (denoted respectively by P, I and D). In practice, the PID controller applies a responsive correction to the controlled function.

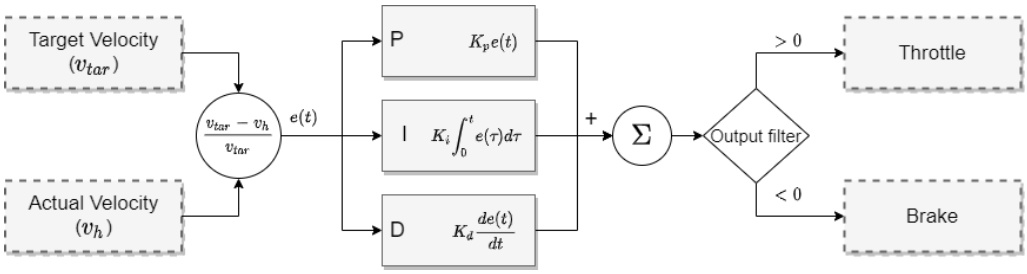


FIGURE 4.2: PID controller for the ACC system

4.2 Design space exploration w.r.t. PID parameters

When designing a PID controller, it is important to understand how to improve the system's performance. As a rule of thumb, by increasing the proportional gain (K_P) we achieve a proportional increase of the control signal for the same level of $e(t)$. In this way, the system reacts more quickly, but it also tends to exceed its target more (overshoot). The parameter of derivative control (K_D) affects the capability of damping, which is important to decrease overshoot. Finally, the value of integral parameter (K_I) affects the capability to limit the steady error, but the system may become more sluggish and oscillatory, if it is not properly tuned.

The runtime verification approach proposed in this thesis can help to adjust the gains K_P, K_I, and K_D by studying their effects in complex driving scenarios with the aim to eventually achieve a satisfactory overall response. We present here the experimental results obtained for two different sets of parameters values:

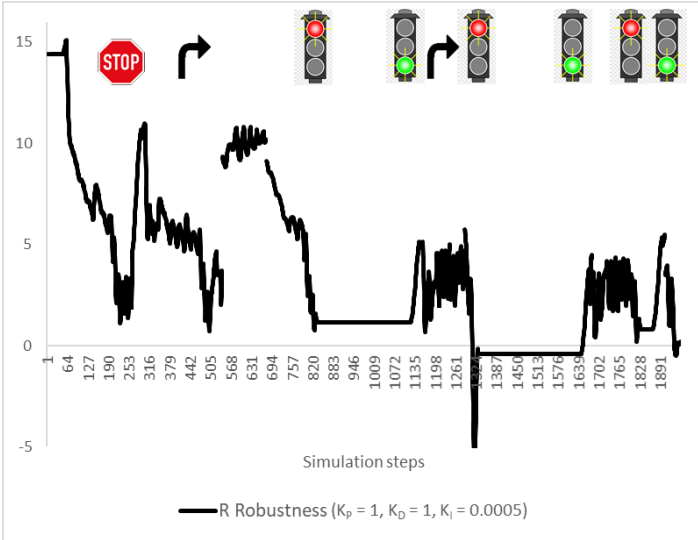
$$K_{P_A} = 1,\ K_{I_A} = 1,\ K_{D_A} = 0.0005$$
$$K_{P_B} = 2,\ K_{I_B} = 0.01,\ K_{D_B} = 0.4$$

Since the primary aim of the ACC system is to keep the safe distance from the preceding vehicle, we evaluate the requirement,

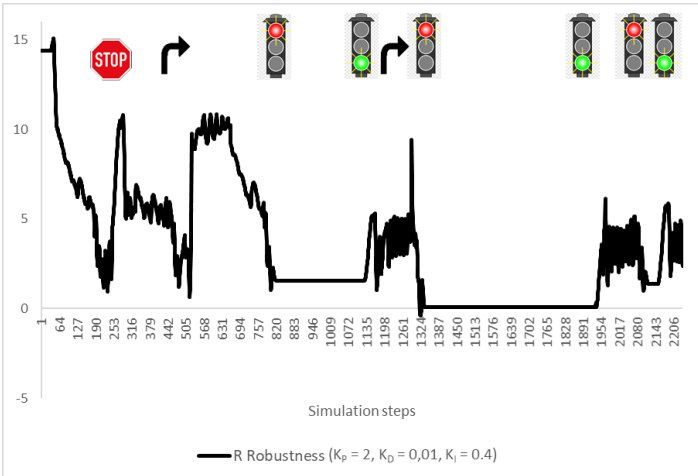
$$R := d - SD >= 0$$

in order to find the parameter set that seems to be more appropriate. For the runtime verification of R and for all other experiments that are reported henceforward, we employed the `rtamt` library to generate discrete-time monitors that were integrated with the CARLA simulation of our ACC system. We run all the experiments at the 3rd Town of CARLA.

Figure 4.3 shows the robustness of R, when monitoring similar driving scenarios with the two aforementioned sets of parameters (A and B). These scenarios refer to the route of two cars moving one behind the other in the CARLA urban environment



(a)



(b)

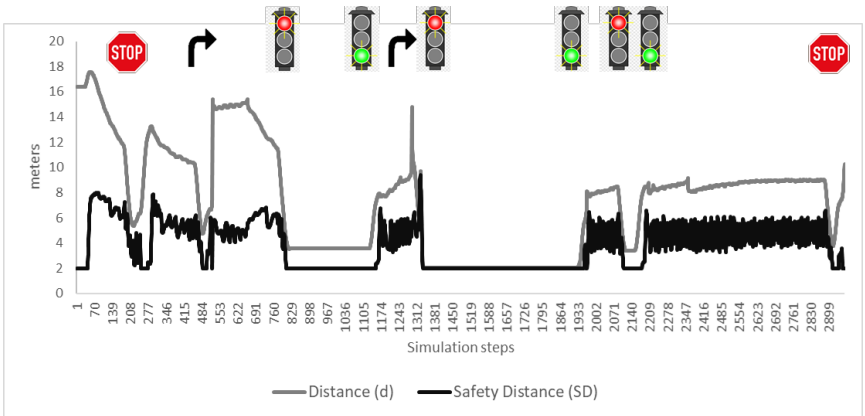
FIGURE 4.3: Robustness of R when the PID is tuned with (a)$K_{P_A}, K_{I_A}, K_{D_A}$ and (b) $K_{P_B}, K_{I_B}, K_{D_B}$

with junctions, stop signs and traffic lights. While following the same route, the scenarios differ slightly with respect to the duration of the red traffic lights.

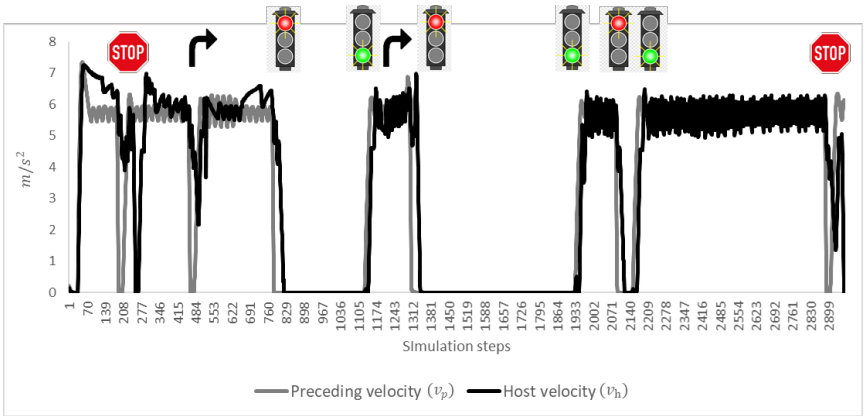
In Figure 4.3a, no robustness value is shown for specific simulation steps, which indicates that the preceding vehicle was too far and, thus, the ACC of the host vehicle was disabled. We observe that when using the parameters $K_{P_A}, K_{I_A}, K_{D_A}$, the system strongly violates R, when the cars stop at the second traffic light, and continues to violate it (with a smaller difference between d and SD) for more than 300 simulation steps, while the cars wait for the green traffic light. On the other hand, when using the parameters $K_{P_B}, K_{I_B}, K_{D_B}$ (Figure 4.3a), there is only a slight violation of R at the same point in time, but the system adjusts quickly and the host vehicle stops at the traffic light maintaining the safety distance.

A more detailed view of the vehicles' performance for the same scenario, extended by additional steps, is presented in Figure 4.4. The graphs show the distance between the host and the preceding vehicle and how it compares with the safety distance (Figure 4.4a), and the velocities of the same two vehicles (Figure 4.4b). From these two graphs, it is easier to interpret the runtime monitoring results. In a stop

signal, the host car stops when the preceding car has already left and similarly in a traffic light the vehicles stop and start moving with a time difference. In Figure 4.4a we can see that the distance d of the two cars gets closer to the safety distance SD as the PID controller adjusts by using more historic data.



(a)



(b)

FIGURE 4.4: (a) Distance between host and preceding vehicle and (b) their velocities

Through experimenting with more similar scenarios, we observed that parameters $K_{P_A}, K_{I_A}, K_{D_A}$ cause violations of R at the second or third traffic light and every next time the vehicles stop for a long period of time. The robustness at another example scenario where this can be clearly observed is presented in Figure 4.5. In this scenario the cars come across four traffic lights. We can clearly observe that at the third and fourth traffic light the host vehicle stops at a distance lower than the safety distance. This indicates that these parameters cannot be used, as the target distance is exceeded and therefore we observe overshooting. To correct this behavior we can increase the K_P parameter to make the host vehicle reacts faster at the deceleration of the preceding vehicle and increase K_D to reduce overshoot. Additionally, we might decrease K_I, therefore reducing how much the system is affected by past values of $e(t)$, i.e. the difference of the safety and the real distance.

When using $K_{P_B}, K_{I_B}, K_{D_B}$ this problem is addressed, as we observed by executing multiple scenarios. For instance for the complex scenario presented in Figure 4.6, there are only some minimal violations that also last short in simulations steps, such as the one at the steps close to 3030. Therefore, in the following analysis we present experimental results obtained using K_{P_B}, K_{I_B} and K_{D_B}.

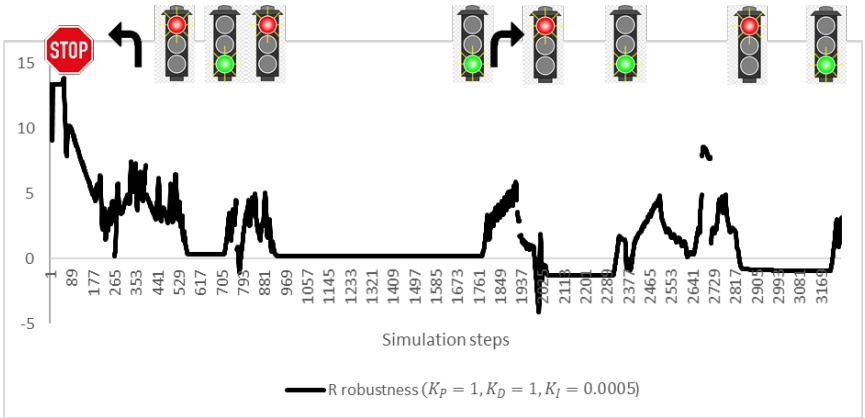


FIGURE 4.5: Robustness of R when the PID is tuned with $K_{P_A}, K_{I_A}, K_{D_A}$ on a more complex scenario

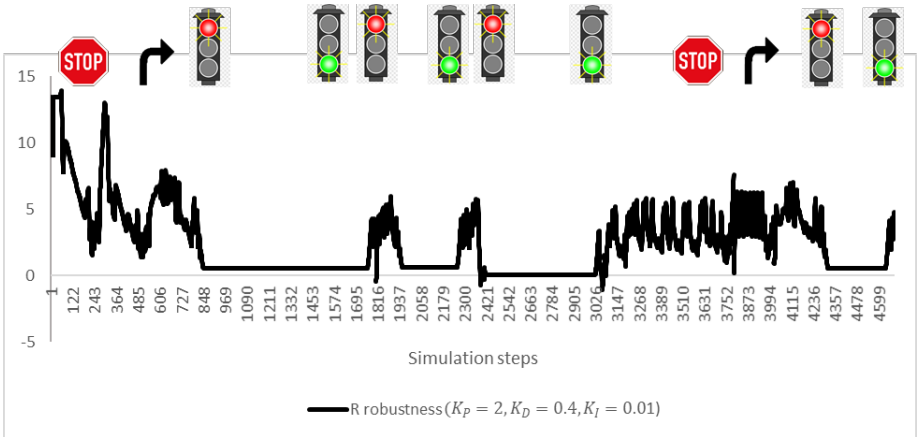


FIGURE 4.6: Robustness of R when the PID is tuned with $K_{P_B}, K_{I_B}, K_{D_B}$ on a more complex scenario

4.3 Requirements for an ACC system

According to [37], any ACC system has to fulfill the following performance requirements, in order to be used in public roads:

R_1. Relative difference $v_p - v_h$ should not be too high, except when the preceding vehicle is out of range or the host vehicle has decelerated, due to a stop signal or a red traffic light, while the preceding vehicle is moving ahead.

R_2. Acceleration of the host vehicle (a_h) must be greater than or equal to G, except if the preceding vehicle's acceleration (a_p) is less than this limit:

$$G := -0.25 \cdot \frac{T_{hw} + 1}{m}$$

where m is the vehicle's mass and T_{hw} is a constant time headway, i.e. constant distance between the two vehicles measured in time that expresses the degree to which the safety distance varies proportionally to the vehicles velocity (spacing policy).

To ensure that our system fulfills these requirements we need to write them as STL properties and monitor them in realistic driving scenarios in CARLA. The STL

property for the first mentioned requirement is:

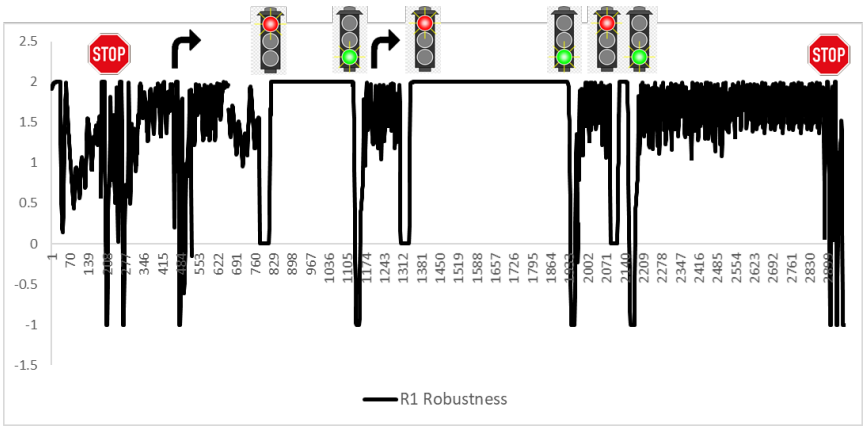
$$R_1 := v_p - v_h < c \lor d > r \lor stopped_h$$

where c is a threshold for the relative difference between velocities, r is the distance threshold under which the ACC system is enabled and $stopped_h$ is `true`, when the host vehicle decelerates, due to a stop signal or a red traffic light. We set the threshold for the difference between the velocities $c = 1.5\, m/s^2$.

The STL property for the second mentioned requirement is:

$$R_2 := a_h \geq G \lor \Box_{[0:t]} a_p < G$$

where $\Box$ is the *always* operator and $[0 : t]$ is the time interval for the host vehicle to adjust its acceleration back to normal, after an extreme deceleration of the preceding vehicle.



(a)

(b)

FIGURE 4.7: Robustness of requirement (a) R_1 and (b) R_2

Figure 4.7a shows the robustness of property R_1 for the scenario described in Figure 4.4. We observe violations after the stop signals and the green traffic lights. Stop signals affect the car velocities in different time steps and, therefore, cause violations. When the host vehicle stops at the signal, the preceding vehicle has already moved ahead, thus causing a excessive difference in their velocities. Similarly, after a traffic light, for which the vehicles are stopped, when it turns green, it takes more steps for the host vehicle to start accelerating in comparison to the preceding vehicle

and the difference in their velocities is again increased. These violations represent the performance variance of our ACC system, due to usual events in urban driving scenarios and, thus, they do not invalidate the performance of our system with respect to R_1. Besides these violations, the requirement is satisfied and the robustness raises as the time proceeds, indicating that the ACC of the host vehicle adjusts to the acceleration changes of the preceding vehicle.

Figure 4.7b shows the robustness of property R_2 for the same scenario. In this experiment, we have set the constant time headway $T_{hw} = 1.5\,sec$, $t = 3$ and the vehicle's mass $m = 1200\,kg$ (mass of a "Seat Leon" that is the host vehicle in all our experiments). We can see that our system fulfills the requirement throughout the scenario, although in some steps the robustness is really low indicating that the host vehicle's acceleration is close to G. Increasing/decreasing T_{hw} we would observe changes on the robustness signals as we require the system to provide a longer/shorter time headway. Doing this, we could examine the lowest time headway for our system to still fulfill the requirement or require a higher time headway for which the robustness is increased.

Chapter 5

Conclusions and Future Work

In order to assure safety for Autonomous Vehicles, the number of conditions to be evaluated scales quickly to millions and, thus, an immense amount of time would be required to complete the validation of a self-driving system if we solely rely on road testing. This emphasizes the need of including simulations in the design loop of autonomous driving systems. With simulation, it's also possible to create outlier scenarios, that can rarely happen on the real world and test them without the safety risks and high costs of physical testing.

Despite the importance of the verification of the simulated systems, little has been done for the exploration of such techniques. An autonomous driving system, being a multi-agent system, continuously interacts with its environment and thus cannot be validated offline, since any change to the software of planning and control would require a new data collection of the same magnitude — clearly unwieldy. Additionally, developing a system through data invariably suffers from lack of transparency, interpretability, and explainability of the actions being taken. Consequently, a runtime verification technique, which is able to provide a robustness level of the designed system throughout the simulation, is required.

In this thesis, we presented an approach for integrating CARLA simulations with runtime monitors generated by the recently presented `rtamt` library. Our proposal enables the validation of autonomous driving control systems through online monitoring STL requirement specifications over realistic driving scenarios.

This development allows to avoid manual inspection of simulation traces or the need to implement ad-hoc test oracles for trace post-processing. Validation of STL requirements over real-world driving scenarios is a valuable means for design space exploration and has the potential to uncover undesired behaviours or to check important performance requirements for the autonomous driving system. To this respect, we showed the results from applying our runtime verification approach for two different purposes. First, to find appropriate parameters for the PID control of an experimental ACC system and second, for checking our ACC system against important performance requirements expressed in STL.

However, this is only a first step towards the set future goals. Firstly, it is crucial to introduce further contributions to allow testing and verification of robustness of autonomous driving systems with machine learning and neural network components. Using runtime monitoring in such systems can result in an increasing explainability and provide indications for the simulation moments that the developed AI models need further improvement. Secondly, another key area of focus should be on developing autonomous safety standards which the industry can use to measure AI competency, thus providing a common metric among all the designed systems.

In addition, the need for even more scenarios is still of paramount importance. This includes developing and deploying new simulation capabilities in the hope of better preparing future vehicles and autonomous vehicle platforms for the real

world. Thus it allows meaningful and systematic scenario testing that covers as much as possible all the potential situations that the future self-driving cars will come across with. To this end, we need to interpret in our approach automating property falsification techniques, that can showcase "intersting" scenarios for which the current self-driving techniques need further improvement. A promising technique would be to combine CARLA, or the other simulators, with software such as Scenic [18], to make the simulator choose in every step the scenario with the lowest predicted robustness, as an "interesting" case that needs exploration.

While CARLA already supports the simulation of such capabilities, we need to further develop our runtime verification approach towards parameter synthesis, falsification, sensitivity analysis and systematic scenario testing. As noted in Section 3.3, these challenges have been addressed when processing Simulink traces, but their application to real-world driving scenarios with street signs, traffic lights, pedestrians and other actors is not obvious.

Appendix A

Installation of CARLA

Instructions for the installation and build of CARLA can be also found in the simulator's documentation page [10] for Windows and Linux. The following instructions are written for the CARLA version 0.9.6.

For the installation Python 3 and the python packages; `pygame` and `numpy` are required. In addition, it is recommended to install on Ubuntu 16.

If you just want to interact with the simulator's world through a client app, without changing the backend of CARLA you can download the latest release, as described in section A.1. If you need to build CARLA locally on your machine, you can follow the instructions mentioned in section A.2.

A.1 Launch the simulator

First of all, download the latest release from CARLA's GitHub page [11] and extract the contents of the repository. In the extracted folder the executable file `CarlaUE4.sh` can be found. Executing this file will launch a window with a view over the city of the simulator. You can wander around the city using the mouse and WASD keys. To get a panoramic view of the city you can point your mouse to the sky, move to that point and, then, point to the city again. This is particularly useful at the begging of the simulation to easily observe the position of the spawned vehicles and pedestrians.

A.2 Build on Linux

Install build tools and dependencies.

```
sudo apt-get update
sudo apt-get install wget software-properties-common
sudo add-apt-repository ppa:ubuntu-toolchain-r/test
wget -O - https://apt.llvm.org/llvm-snapshot.gpg.key|sudo apt-key add -
sudo apt-add-repository "deb␣http://apt.llvm.org/xenial/␣llvm-toolchain-
    xenial-7␣main"
sudo apt-get update
sudo apt-get install build-essential clang-7 lld-7 g++-7 cmake ninja-build
     libvulkan1 python python-pip python-dev python3-dev python3-pip
    libpng16-dev libtiff5-dev libjpeg-dev tzdata sed curl unzip autoconf
    libtool rsync
pip2 install --user setuptools
pip3 install --user setuptools
```

It is recommended to change your default clang version to compile Unreal Engine and the CARLA dependencies to avoid compatibility issues.

```
sudo update-alternatives --install /usr/bin/clang++ clang++ /usr/lib/llvm
    -7/bin/clang++ 170
sudo update-alternatives --install /usr/bin/clang clang /usr/lib/llvm-7/
    bin/clang 170
```

You also need to install Unreal Engine 4.22 (UE4). [1] To gain access to the repository you need to signup at their webpage [41] and link it to your GitHub account (this can be done through the UE4 webpage in Settings/Connected Accounts). The commands bellow assume that you clone the Unreal Engine repository to the folder `~\UnrealEngine_4.22`. You can choose your preferred folder by accordingly changing the paths.

```
git clone --depth=1 -b 4.22 https://github.com/EpicGames/UnrealEngine.git
    ~/UnrealEngine_4.22
cd ~/UnrealEngine_4.22
./Setup.sh && ./GenerateProjectFiles.sh && make
```

Note that the minimum system requirements for UE4 are; Desktop PC or MAC (64-bit architecture), Quad-core Intel or AMD processor, 2.5 GHz or faster, NVIDIA GeForce 470 GTX or AMD Radeon 6870 HD series card or higher, 8 GB RAM.

Finally, you have to clone the project from the GitHub repository of CARLA [11]. After cloning, you need to download the assets package and set an environment variable for the Unreal Engine.

```
./Update.sh
export UE4_ROOT=~/UnrealEngine_4.22
```

Now that the environment is set up, you can use make to run different commands and build the different modules.

```
make launch       # Compiles the simulator and launches the Unreal Engine
                    Editor.
make PythonAPI    # Compiles the PythonAPI module.
make package      # Compiles everything and creates a packaged version able
                    to run without the UE4 editor.
make launch-only  # Launches the Unreal Engine Editor without compiling.
make help         # Prints all available commands.
```

Having launched the simulator, you can choose a city and hit play to run CARLA.

A.3 Execute python scripts

After the simulator is launched (either by launching Unreal Engine through the make command or by executing the `CarlaUE4.sh` contained in the pre-compiled version), you can navigate at the empty city. To spawn cars and pedestrians, get simulation data and apply commands you can run a python script to connect to the backend.

To do so, simply execute the python file from the same or a remote machine. The configuration with the server is done inside the script, as explained in section 2.2.1.

```
python my-script.py
```

[1] At the time this thesis was written CARLA was not compatible with newer versions of Unreal Engine.

You can use many clients for the same simulation, while each one of them can have control on all the actors, even on the ones spawned by another client. Also the actors spawned by one client will remain active at the simulation, if not destroyed, even if this client disconnects.

Appendix B

Signal Temporal Logic (STL) syntax

We provide here the reader with some background on STL [26], and in particular the syntax with *past* and *future* operators. STL is an extension of the propositional Linear-time Temporal Logic (LTL), which allows writing property specifications for behaviours that unfold in continuous (dense) time and using predicates on numerical values (as opposed to using only atomic propositions in LTL).

Let X be a set of real-valued variables and a valuation $v : X \to \mathbb{R}$ that for $x \in X$ maps a variable $x \in X$ to a real value. We denote by $V(X)$ the set of valuations over variables in X. A *signal* w defined over a *finite* time domain $\mathbb{T} = [0, n]$ and the set of variables X is a sequence $v_0 \cdot v_1 \cdots v_n$ of valuations over X. We denote by $|w| = n + 1$ the *length* of signal w.

The syntax of a STL formula φ over X is defined by the following grammar:

$$\varphi := f(X) \sim c \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \, \mathcal{U}_I \, \varphi_2 \mid \varphi_1 \, \mathcal{S}_I \, \varphi_2$$

where $\sim \in \{<, \leq\}$, $c \in \mathbb{R}$ and I is of the form $[a, b]$ or $[a, \infty)$ such that a and b are in $\mathbb{N}$ and $0 \leq a \leq b$.

STL, apart from its boolean semantics, also features a quantitative semantics. Let φ be an STL formula and w a signal trace. The robustness degree does not only report the satisfaction/violation of property φ, but it also gives us a measure of whether w satisfies it by far or very marginally. We define the robustness $\rho(\varphi, w, t)$, over discrete time, by induction, as follows:

$$\begin{aligned}
\rho(true, w, t) &= +\infty \\
\rho(f(X) \sim c, w, t) &= f(w_X[t]) - c \\
\rho(\neg\varphi, w, t) &= -\rho(\varphi, w, t) \\
\rho(\varphi_1 \vee \varphi_2, w, t) &= \max\{\rho(\varphi_1, w, t), \rho(\varphi_2, w, t)\} \\
\rho(\varphi_1 \, \mathcal{U}_I \, \varphi_2, w, t) &= \sup_{t' \in (t \oplus I) \cap \mathbb{T}} \min \left\{ \begin{array}{c} \rho(\varphi_2, w, t'), \\ \inf_{t'' \in (t, t')} \rho(\varphi_1, w, t'') \end{array} \right\} \\
\rho(\varphi_1 \, \mathcal{S}_I \, \varphi_2, w, t) &= \sup_{t' \in (t \ominus I) \cap \mathbb{T}} \min \left\{ \begin{array}{c} \rho(\varphi_2, w, t'), \\ \inf_{t'' \in (t', t)} \rho(\varphi_1, w, t'') \end{array} \right\}
\end{aligned}$$

where $\oplus$ and $\ominus$ are Minkowski sum and difference. if $\rho(\varphi, w, t) > 0$ then the signal w satisfies the property φ, while if $\rho(\varphi, w, t) < 0$ then the signal w violates the property φ. From this basic definition of STL we use standard rules to derive the

other Boolean and temporal operators:

true	true	$\equiv$	$\varphi \vee \varphi'$	false	false	$\equiv$	$\neg \mathit{true}$
conjunction	$\varphi_1 \wedge f_2$	$\equiv$	$\neg(\neg\varphi_1 \vee \neg\varphi_2))$	eventually	$\Diamond_I\, \varphi$	$\equiv$	$\mathit{true}\, \mathcal{U}_I\, \varphi$
always	$\Box_I\, \varphi$	$\equiv$	$\neg\Diamond_I\, \neg\varphi$	once	$\diamondminus_I\, \varphi$	$\equiv$	$\mathit{true}\, \mathcal{S}_I\, \varphi$
historically	$\boxminus_I\, \varphi$	$\equiv$	$\neg\diamondminus_I\, \neq \varphi$	next	$\bigcirc\, \varphi$	$\equiv$	$\mathit{false}\, \mathcal{U}_{(0,\infty)}\, \varphi$
previous	$\ominus\, \varphi$	$\equiv$	$\mathit{false}\, \mathcal{S}_{(0,\infty)}\, \varphi$	rise	$\uparrow \varphi$	$\equiv$	$\ominus\, \neg\varphi \wedge \varphi$
fall	$\downarrow \varphi$	$\equiv$	$\ominus\, \varphi \wedge \neg\varphi$				

Appendix C

Python script fot an rtamt Monitor

C.1 Initialize the monitor with the required specifications:

Below we present an example script used to initialize a monitor for the condition:

$$\Box(d - SD >= 0)$$

```
spec = rtamt.STLIOSpecification(1)
spec.name = 'Monitor'

spec.declare_var('dist', 'float')
spec.declare_var('SD', 'float')
spec.declare_var('rob', 'float')

spec.ste_var_io_type('dist', rtamt.StlIOType.OUT)
spec.ste_var_io_type('SD', rtamt.StlIOType.IN)
spec.ste_var_io_type('rob', rtamt.StlIOType.OUT)

spec.spec = 'rob = always((dist - SD >= 0))'

try:
  spec.parse()
except rtamt.STLParseException as err:
  print('STL Parse Exception: {}'.format(err))
  sys.exit()
```

C.2 Online monitoring for three events A, B and C:

```
# after event A (distance = distA, SD = SDA) occurs at timestamp tA
rob = spec.update(tA, [('dist', distA), ('SD', SDA)])
print('The robustness at timestamp: {} is {} '.format(tA, rob))

# after event B (distance = distB, SD = SDB) occurs at timestamp tB
rob = spec.update(tB, [('dist', distB), ('SD', SDB)])
print('The robustness at timestamp: {} is {} '.format(tB, rob))

# after event C (distance = distC, SD = SDC) occurs at timestamp tC
rob = spec.update(tB, [('dist', distC), ('SD', SDC)])
print('The robustness at timestamp: {} is {} '.format(tC, rob))
```

Let's assume the events presented in C.1. The output would be:

```
The robustness at timestamp: 1.11 is -0.1
The robustness at timestamp: 1.16 is -0.1
The robustness at timestamp: 1.21 is -0.2
```

TABLE C.1: A, B, C events for online monitoring.

event	timestamp	distance	Safety Distance
A	1.11	3.4	3.5
B	1.16	3.7	3.5
C	1.21	3.3	3.5

C.3 Offline monitoring:

```
for i in range(len(events))
  rob = spec.update(i, [('dist', events[i]['dist']), ('SD', events[i]['SD'
      ])])

print('The final robustness is: {} '.format(rob))
```

For the events of table C.1, the output would be:

```
The final robustness is: -0.2
```

Bibliography

[1] *AirSim*. https://microsoft.github.io/AirSim/.

[2] Rajeev Alur and Thomas A. Henzinger. "Logics and models of real time: A survey". In: *Real-Time: Theory in Practice*. Springer, 1992, pp. 74–106.

[3] Yashwanth Annpureddy et al. "S-TaLiRo: A Tool for Temporal Logic Falsification for Hybrid Systems". In: *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2011, pp. 254–257.

[4] *Autonomous Vehicles Cannot Be Test-Driven Enough Miles to Demonstrate Their Safety; Alternative Testing Methods Needed*. https://www.rand.org/news/press/2016/04/12.html. Apr. 2016.

[5] *Autoware.AI*. https://www.autoware.ai/.

[6] Ezio Bartocci et al. "Specification-Based Monitoring of Cyber-Physical Systems: A Survey on Theory, Tools and Applications". In: *Lectures on Runtime Verification: Introductory and Advanced Topics*. Springer, 2018, pp. 135–175.

[7] S. Bechtold and Bernhard Höfle. "HELIOS: A MULTI-PURPOSE LIDAR SIMULATION FRAMEWORK FOR RESEARCH, PLANNING AND TRAINING OF LASER SCANNING OPERATIONS WITH AIRBORNE, GROUND-BASED MOBILE AND STATIONARY PLATFORMS". In: *ISPRS Annals of Photogrammetry, Remote Sensing and Spatial Information Sciences* III-3 (June 2016), pp. 161–168. DOI: 10.5194/isprsannals-III-3-161-2016.

[8] *C++ CARLA documentation*. http://carla.org/Doxygen/html/index.html.

[9] *Carcraft*. https://www.theatlantic.com/technology/archive/2017/08/inside-waymos-secret-testing-and-simulation-facilities/537648/.

[10] *CARLA documentation*. https://carla.readthedocs.io/en/latest/.

[11] *CARLA github repository*. https://github.com/carla-simulator/carla/blob/master/Docs/download.md.

[12] Edmund Clarke, Orna Grumberg, and Doron Peled. *Model Checking*. Jan. 2001. ISBN: 978-0-262-03270-4.

[13] *DeepDrive*. https://deepdrive.voyage.auto/.

[14] Alexandre Donzé. "Breach, A Toolbox for Verification and Parameter Synthesis of Hybrid Systems". In: *Computer Aided Verification*. Springer, 2010, pp. 167–170.

[15] Alexandre Donzé and Oded Maler. "Robust Satisfaction of Temporal Logic over Real-Valued Signals". In: *Formal Modeling and Analysis of Timed Systems*. Springer, 2010, pp. 92–106.

[16] Alexey Dosovitskiy et al. "CARLA: An Open Urban Driving Simulator". In: *Proceedings of the 1st Annual Conference on Robot Learning*. 2017, pp. 1–16.

[17] Tommaso Dreossi et al. "VerifAI: A Toolkit for the Formal Design and Analysis of Artificial Intelligence-Based Systems". In: *Computer Aided Verification*. Springer, 2019, pp. 432–442.

[18] Daniel J. Fremont et al. "Scenic: a language for scenario specification and scene generation". In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2019* (2019). DOI: 10.1145/3314221.3314633. URL: http://dx.doi.org/10.1145/3314221.3314633.

[19] Alwyn Goodloe and Lee Pike. *Monitoring Distributed Real-Time Systems: A Survey and Future Directions*. 2010.

[20] Xiaoqing Jin et al. "Powertrain Control Verification Benchmark". In: *Proc. of 17th Int. Conf. on Hybrid Systems: Computation and Control*. HSCC '14. Berlin, Germany: Association for Computing Machinery, 2014, pp. 253–262.

[21] James Kapinski et al. "ST-Lib: A Library for Specifying and Classifying Model Behaviors". In: *SAE Technical Paper*. SAE International, 2016.

[22] Martin Leucker and Christian Schallhart. "A Brief Account of Runtime Verification". In: *Journal of Logic and Algebraic Programming* 78 (May 2009), pp. 293–303. DOI: 10.1016/j.jlap.2008.08.004.

[23] *LGSVL Simulator*. https://www.lgsvlsimulator.com/.

[24] Rupak Majumdar et al. "Paracosm: A Language and Tool for Testing Autonomous Driving Systems". In: *CoRR* abs/1902.01084 (2019). arXiv: 1902.01084. URL: http://arxiv.org/abs/1902.01084.

[25] Oded Maler and Dejan Nickovic. "Monitoring Temporal Properties of Continuous Signals". In: *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*. Springer, 2004, pp. 152–166.

[26] Oded Maler and Dejan Ničković. "Monitoring Temporal Properties of Continuous Signals". In: *Proc. of Joint Int. Conferences on Formal Modelling and Analysis of Timed Systems*. Vol. 3253. LNCS. Springer, 2004, pp. 152–166.

[27] J. Mao and L. Chen. "Runtime Monitoring for Cyber-physical Systems: A Case Study of Cooperative Adaptive Cruise Control". In: *2012 Second International Conference on Intelligent System Design and Engineering Application*. 2012, pp. 509–515.

[28] Valeriano Méndez et al. "SIMLIDAR – Simulation of LIDAR performance in artificially simulated orchards". In: *Biosystems Engineering - BIOSYST ENG* 111 (Jan. 2012). DOI: 10.1016/j.biosystemseng.2011.10.010.

[29] *NetworkX; python package for complex networks*. https://networkx.github.io/.

[30] Dejan Nickovic and Tomoya Yamaguchi. "RTAMT: Online Robustness Monitors from STL". In: *CoRR* abs/2005.11827 (2020). arXiv: 2005.11827. URL: https://arxiv.org/abs/2005.11827.

[31] *OpenDRIVE file format*. http://opendrive.org/.

[32] *Radar Simulator (RADSim)*. https://www.riversideresearch.org/.

[33] Francisca Rosique et al. "A Systematic Review of Perception System and Simulators for Autonomous Vehicles Research". In: *Sensors* 19 (Feb. 2019), p. 648. DOI: 10.3390/s19030648.

[34] Shai Shalev-Shwartz, Shaked Shammah, and Amnon Shashua. "On a Formal Model of Safe and Scalable Self-driving Cars". In: *CoRR* abs/1708.06374 (2017). arXiv: 1708.06374. URL: http://arxiv.org/abs/1708.06374.

[35] *SimSonic*. http://www.simsonic.fr/.

[36] *Simulation is key to real-world autonomous driving*. https://www.theengineer.co.uk/autonomous-vehicles-simulation-real-word-road/. May 2020.

[37] Taku Takahama and Daisuke Akasaka. "Model Predictive Control Approach to Design Practical Adaptive Cruise Control for Traffic Jam". In: *International journal of automotive engineering* 9 (2018), pp. 99–104.

[38] C. E. Tuncali et al. "Requirements-Driven Test Generation for Autonomous Vehicles With Machine Learning Components". In: *IEEE Transactions on Intelligent Vehicles* 5.2 (2020), pp. 265–280.

[39] *Udacity self-driving car simulator*. `https://medium.com/activating-robotic-minds/introduction-to-udacity-self-driving-car-simulator-4d78198d301d`.

[40] *Unity*. `https://unity3d.com`.

[41] *Unreal Engine*. `https://www.unrealengine.com/en-US/`.

[42] "What's Decidable about Hybrid Automata?" In: *Journal of Computer and System Sciences* 57.1 (1998), pp. 94 –124.

[43] John Woods and John Christian. "Glidar: An OpenGL-based, Real-Time, and Open Source 3D Sensor Simulator for Testing Computer Vision Algorithms". In: *Journal of Imaging* 2 (Jan. 2016), p. 5. DOI: `10.3390/jimaging2010005`.